

Project Document

Model Rocket Project



Prepared by:

Emmanuel Basua

Prepared for:

EEE1000X EEE3000X

Department of Electrical Engineering

University of Cape Town

version 0.7

August 4, 2026

Contents

1	Introduction	1
1.1	Problem Description	1
1.2	Background	1
1.3	Scope and Limitations	2
1.3.1	Scope	2
1.3.2	Limitations	2
2	Requirements Analysis	3
2.1	Requirements	3
2.2	Specifications	3
2.2.1	Acceptance Testing Procedures	4
3	Objectives	6
3.1	12 week outline	6
4	Flight Computer	8
4.1	Microcontroller	8
4.2	Sensors	9
4.2.1	Accelerometer	9
4.2.2	Barometric Pressure Sensor	10
4.2.3	GPS Module	12
4.3	Power circuit	15
4.3.1	Voltage Regulation	15
4.3.2	Battery	16
4.4	Telemetry Computer	18
4.4.1	Telemetry Microcontorller	18
5	Ignition and Pyro System	19
5.1	Ignition System	19
5.1.1	MOSFETs	19
5.2	Pyro System	20
5.2.1	Overview	20
5.2.2	Ejection Charge Ignition System	20
5.2.3	Parachute Configuration	22
5.3	Motor Selection Overview	23
6	CAD Design	26
6.1	Overview	26

6.1.1	Full Assembly	26
6.1.2	OpenRocket Simulation	27
6.1.3	Nose Cone	27
6.1.4	Body Tube	28
6.1.5	Electronics Bay	28
6.1.6	Fin Assembly	29
6.1.7	Motor Mount	29
6.1.8	Launch Lug	30
6.1.9	Launch Pad Assembly	30
7	Ground Station Design	33
7.1	Ground Station Architecture Overview	33
7.2	Ground Station Hardware	33
7.2.1	Microcontroller / Computer Selection: Raspberry Pi 4	33
7.3	Software Stack	34
7.3.1	Dependencies	34
7.3.2	MQTT Topic Architecture	35
7.3.3	Main Application: ground_station_app.py	36
7.3.4	Flask REST API Endpoints	38
7.3.5	Data Logging	39
7.3.6	Systemd Auto-Start Service	39
7.4	Web Dashboard Interface	40
7.4.1	Status Bar	40
7.4.2	Telemetry Data Panel	40
7.4.3	Flight Statistics Panel	41
7.4.4	GPS Map	42
7.4.5	Altitude and Acceleration Graphs	42
7.4.6	Flight Events Log	42
7.4.7	Control Panel	43
7.5	Network Configuration	43
7.5.1	WiFi Access Point	43
7.5.2	Mosquitto MQTT Broker Configuration	44
8	Additional	45
8.1	Bill of Materials	45
8.2	Acceptance Test Results	47
8.3	System Integration and Interfacing	48
8.4	Discussion and Considerations	48
8.4.1	BMP280 Failures and Shortcomings	48

Chapter 1

Introduction

1.1 Problem Description

The task at hand is to design a model rocket or low power rocket that can reach up to 1km. That is robust enough to transmit flight data and safely land from various altitudes whilst also being easily recoverable. Additionally the rocket should be designed to facilitate usage by non technical users and easily replicable.

1.2 Background

Model rockets are lightweight, reusable rockets designed for controlled, safe experimentation and do not require a license to operate. They are powered by solid rocket engines and built with non-metallic airframes. Model rocketry offers an accessible introduction to aerospace and electrical engineering, allowing hands-on learning about aerodynamics, propulsion, electronics, and flight mechanics.

The ignition system, typically an electrical igniter, is responsible for safely starting a rocket motor. An ignition computer or flight computer automates this process, waiting for safe launch conditions, sending current to the igniter, and recording telemetry data and flight events. Advanced ignition systems can handle multiple-stage launches, safety interlocks, or countdowns.

A telemetry computer is an electronic system that monitors and transmits real-time data during a rocket's flight, including altitude, acceleration, velocity, orientation, and system status. It aids in flight performance analysis, safety tracking, and engineering validation. Telemetry units often use RF transmitters, Bluetooth, or LoRa to send data to a ground station, ensuring accurate and timely data transmission.

A ground station receives real-time telemetry data from a rocket, displays flight parameters like altitude, velocity, acceleration, battery voltage, and GPS position, aids in recovery by tracking landing location, monitors system status like pyro continuity, arming state, and apogee trigger events, and logs data for post-flight analysis.

Model rockets use solid rocket motors classified by total impulse, with A-D motors being low power, E-G motors mid power and H and up motors high power (requiring certification). Different propellant types include black powder (A-D motors), composite propellant (E and up), reloadable motors (engine casing reused), and single-use motors (entirely disposable).

1.3 Scope and Limitations

1.3.1 Scope

- A rocket that can travel up to 500m
- A flight computer with sensors, data transfer, and ground station communication
- Use of a recovery parachute
- A ground station that can accurately and visibly display flight data
- User friendly and replicable design

1.3.2 Limitations

- Limited rocket engine types class A-E.
- Communication outlets like bluetooth or wifi have a limited range
- Sensor inaccuracy with small rockets gps modules.
- Balancing advanced feature with affordability to allow for easy replication.

Chapter 2

Requirements Analysis

2.1 Requirements

The requirements for a the model rocket are described in [Table 2.1](#).

Table 2.1: User and functional requirements.

Requirement ID	Description
R01	The rocket must have the capacity to reach up to 500 meters (0.5 km) above the ground.
R02	The rocket must continuously gather and send real-time flight data.
R03	The rocket must release a parachute recovery system..
R04	All internal electronic circuitry must remain intact after landing.
R05	Ground station must display real-time flight data.
R06	The design must be easily replicated with reasonably priced, commercially available components.
R07	The rocket shall be easily recoverable and maintainable for repeated use.
R08	Every electronic equipment must function dependably under all climatic and flight envelope circumstances..

2.2 Specifications

The specifications, refined from the requirements in [Table 2.1](#), for the model rocket are described in [Table 2.2](#).

Table 2.2: Specifications of the sensing subsystem derived from the requirements in Table 2.1.

Specification ID	Description
SP01	Target altitude of 500m $\pm$ 50m.
SP02	Minimum 10Hz sampling rate.
SP03	Altitude, acceleration (3-axis), velocity, orientation, battery voltage, GPS position.
SP04	Onboard memory backup with minimum 1000 data points capacity.
SP05	RF, Bluetooth, or LoRa communication.
SP06	Automatic detection of the apogee.
SP07	Maximum deceleration of 10G upon landing.
SP08	Non-metallic airframe (cardboard, fiberglass, or plastic)
SP09	Launch acceleration tolerance of up to 15G.
SP10	Landing impact tolerance of up to 20G.
SP11	Display of data in real time less than 500 ms latency
SP12	Altitude, velocity, and acceleration curves are graphically displayed.
SP13	All flight data is automatically stored in common formats (CSV, JSON).
SP14	Integration of mapping and GPS coordinate display.
SP15	All parts sourced from standard suppliers.
SP16	Constructible with basic tools and soldering skills.
SP17	All parts sourced from standard suppliers.
SP18	Minimum 2-hour operational battery life.
SP19	$\pm 2\%$ for altitude, $\pm 5\%$ for acceleration measurements
SP20	$>90\%$ packet success rate within range.
SP21	Typical landing within 500m radius of launch site.
SP22	Maximum 30 minutes for routine maintenance to allow relaunch.

2.2.1 Acceptance Testing Procedures

Acceptance testing is split into two categories: **Hardware ATPs**, verified through MATLAB numerical simulation, and **Software ATPs**, verified through live bench demonstrations of the flight computer and ground station.

Hardware Acceptance Test Procedures (MATLAB-Simulated)

ID	Description	MATLAB Simulation Procedure	Pass/Fail Criteria
AT-HW01	Ascent trajectory: altitude, velocity, acceleration (R01, SP01, SP09)	Integrate 1-DOF equations of motion using the digitised C6-5 thrust curve, airframe mass, and drag model.	Apogee within 500 m $\pm$ 50 m (or within 10% of Open-Rocket's 219 m); peak acceleration $\leq$ 15 G.
AT-HW02	Static stability margin (Section 5.3)	Compute CG and Barrowman CP across the burn as propellant depletes; plot margin in calibres vs. time.	Margin remains within 1–2 calibres for the entire boost phase.
AT-HW03	Recovery descent, landing impact, and wind drift (SP07, SP10, SP21)	Simulate parachute descent to terminal velocity; run a Monte Carlo wind sweep (0–5 m/s) to estimate landing dispersion and impact deceleration.	Impact deceleration $\leq$ 10 G (nominal), $\leq$ 20 G (electronics limit); 95th-percentile landing radius $\leq$ 500 m.
AT-HW04	Structural safety factors: motor retention & body tube buckling	Compute peak axial thrust load from AT-HW01 output; compare to retention clip holding force and Euler buckling load of the body tube.	Retention safety factor $\geq$ 2; buckling safety factor $\geq$ 1.5.

Table 2.3: Hardware Acceptance Testing Procedures (MATLAB simulation-based)

Software Acceptance Test Procedures (Bench Demonstration)

ID	Description	Demo Procedure	Pass/Fail Criteria
AT-SW01	Sensor sampling rate (SP02)	Serial-monitor demo of live BMP280/MPU6050 sample counter over 60 s.	$\geq$ 10 Hz sustained on both sensors.
AT-SW02	Apogee detection (SP06)	Manually vary pressure at the BMP280 while projecting the FSM state transition live.	FSM transitions to ejection state within 1 s of peak, no false triggers.
AT-SW03	Onboard memory backup (SP04)	Log >1000 points, power-cycle live, dump recovered log over serial.	All pre-loss points recovered, shown on-screen.
AT-SW04	Telemetry latency (SP05, SP11)	Side-by-side publish/receive timestamp display over a packet run.	Median one-way latency <500 ms.
AT-SW07	Data logging format (SP13)	Start/stop logging during a mock stream; open CSV live.	All 16 schema columns populated, no malformed rows.

Table 2.4: Software Acceptance Testing Procedures (bench-demonstrable, reduced set)

Chapter 3

Objectives

3.1 12 week outline

Table 3.1: Interfacing specifications

Week	Objective	Criteria	Due Date
1	Project Document	• A clear description of project and outline of tasks to be completed to achieve the goals of the project.	04/07/2025
2	Ignition System Design Part 1	• Detailed System outline for MCU an Barometric Sensor • Component Selection and Justification • Preliminary System Schematic • Preliminary Testing	14/07/2025
3	Ignition System Design Part 2	• Detailed System outline for Power System • Component Selection and Justification • Preliminary System Schematic • Preliminary Testing	21/07/2025
4	Telemetry Computer Design Part 1	• Detailed System outline Rasp Pi • Component Selection and Justification	16/02/2026
5	Pyro System	• Detailed System outline for E-matches and Parachute • Component Selection and Justification • Parachute Construction and Demo • E-match acquisition	16-19/06/2026
6	CAD Designs	• Detailed outline • Final designs for rocket and launchpad	16-19/06/2026
7	Engine Selection & 3D Printing	• Detailed System outline • Engine Selection and Justification • 3D printed models demo	23-26/06/2026
8	Ground Station Design Part 1 (Ideally a simple app)	• Continuously receive, decode, and interpret data from the rocket in real-time • Display sensor data dynamically during flight	23-26/06/2026
Continued on next page			

Table 3.1 – continued from previous page

Week	Objective	Criteria	Due Date
9	Ground Station Design Part 2	• Save incoming telemetry data to a file for post-flight analysis • Detect and react to flight events	30-3/06- 07/2026
10	Ground Station Design Part 3	• Implement GPS monitoring and integrate it with web dashboard for live location broadcasting • Implement battery monitoring and integrate it with web dashboard • Detect and react to flight events -	7- 10/07/2026
11	All system testing and integration	• Detect and react to flight events • All predetermined tests have been carried out • Results of the test are provided • Interfaces are tested and systems are ready to be integrated	14- 17/07/2026
12	Final Board Construction and Launch	• All module's are ready to be ordered. • Entire system cleanly laid-out on Veroboard per space requirements • Rocket successfully assembled and ready to launch • Rocket successfully/unsuccessfully launched	20- 24/07/2026

Chapter 4

Flight Computer

4.1 Microcontroller

The model rocket's microcontroller serves as its brain. It handles data logging and telemetry transmission, controls systems like ignition and parachute deployment, processes sensor data, and regulates timing. In order to carry out crucial procedures like apogee detection or fail-safes and guarantee a safe and efficient flight, it continuously checks the flight conditions. When considering which microcontroller the following chips stood out; ESP-VROOM-32, STM32F4 and Teensy 4.0.

Teensy 4.0 Justification

The Teensy 4.0's high processing speed (600 MHz), small size, and flexible I/O capabilities make it the perfect microcontroller for a model rocket flight computer. For precise flight tracking and prompt event triggers like ignition and parachute deployment, its quick processor allows real-time data handling for sensors like IMUs and barometers. Numerous digital, analogue, and communication interfaces enable expansion with logging, telemetry, and GPS, and its compact form factor makes it simple to fit into small rocket airframes. It is also a sensible option due to its low power consumption and affordability, providing the functionality of much larger systems in a small, effective package.

STM32F4 Justification

Because of their powerful ARM Cortex-M4 cores (up to 180 MHz with FPU), the STM32F4 series microcontrollers are a good option for model rocket flight computers. Fast GPIO and interrupt handling allow them to provide precise control over ignition and ejection systems, and integrated interfaces such as SPI, I2C, UART, ADC, and timers facilitate high-speed data collection from sensors, logging, and telemetry modules. Accurate timekeeping and effective multitasking are made possible by their integrated hardware features, which include DMA and a real-time clock. STM32F4 MCUs offer superior performance, flexibility, and stability for sophisticated rocket avionics, despite being somewhat larger and more difficult to program than some boards.

Specification Adjusted Comparison

The ESP32 fulfills most of the microcontroller-based specifications because of its incredible WiFi/Bluetooth integration. Thus it is the chosen choice.

Specification ID	Specification	ESP32	Teensy 4.0	STM32F4
SP02	Minimum 10Hz sampling rate	Up to 1kHz	Up to 10kHz+	Up to 10kHz+
SP03	Multi-sensor data collection	18 ADC channels	14 ADC channels	16 ADC channels
SP04	1000+ data points storage	4MB flash	2MB flash	512KB-1MB
SP05	RF/Bluetooth/LoRa comms	Built-in WiFi+BT	External module needed	External module needed
SP11	<500ms real-time latency	WiFi transmission	Need external radio	Need external radio
SP18	2-hour battery life	Low power modes	Higher consumption	Very low power
SP20	>90% packet success	WiFi reliability	No built-in comms	No built-in comms
SP21	500m range capability	WiFi range adequate	Need external radio	Need external radio

Table 4.1: Caption

4.2 Sensors

When selecting the required sensors for the model rocket being a barometric pressure sensor, accelerometer, and a gyroscope. Two options were considered for each category:

4.2.1 Accelerometer

The accelerometer measures linear acceleration along one or more axes. By spotting abrupt changes in acceleration, it assists in the detection of impact, launch, and free-fall events. In addition, it helps with motion tracking, velocity reconstruction, and the detection of anomalies or disruptions in flight.

LSM6DS3TR-C Justification

The LSM6DS3TR-C is a great option for measuring motion and orientation in a small package because it combines a 3-axis gyroscope and a 3-axis accelerometer. At a sampling rate of up to 6.6 kHz, it offers high-resolution data. Digital motion processing (DMP) and a finite state machine are two embedded features of the sensor that assist in transferring computational workloads from the MCU. Its low power consumption, which can be as low as 0.65 mA in high-performance mode, fits within the limitations of the power budget. Software development is made easier by the LSM6DS3TR-C's extensive support for Arduino and ESP32 libraries.

MPU6050 Justification

An affordable and useful solution for motion sensing in model rocketry is the MPU6050, a 6-axis inertial measurement unit (IMU) that combines a 3-axis accelerometer and a 3-axis gyroscope into a single small chip. It has adjustable measurement ranges of $\pm 2g$ to $\pm 16g$ (acceleration) and ± 250 to $\pm 2000^\circ/s$ (gyro) orientation during flight, and it supports sampling rates of up to 1 kHz. Basic sensor fusion and gesture detection are made possible by its internal digital motion processor (DMP), which relieves the main microcontroller of some of the strain. The MPU6050 is still very appealing

for educational or cost-sensitive designs because of its extensive availability, established open-source software support, and demonstrated dependability in aerospace hobbyist applications, even though it is not as feature-rich or power-efficient as more recent alternatives like the LSM6DS3TR-C.

Specification Adjusted Comparison

Specification ID	Specification	MPU6050	LSM6DS3TR-C
SP02	Minimum 10Hz sampling rate	Up to 1kHz	Up to 6.6kHz
SP03	Multi-axis motion sensing	3-axis accel + 3-axis gyro	3-axis accel + 3-axis gyro
SP09	Launch acceleration tolerance	$\pm 2g$ to $\pm 16g$	$\pm 2g$ to $\pm 16g$
SP10	Landing impact tolerance	$\pm 16g$ max	$\pm 16g$ max
SP18	2-hour battery life compatibility	3.9mA active	0.65mA (high-performance)
SP19	Acceleration accuracy	$\pm 5\%$ (typical)	$< \pm 2\%$ (factory trimmed + DMP)
SP06	Apogee detection support	Basic threshold detection	Built-in finite state machine
–	Communication interface	I ² C	I ² C / SPI

Table 4.2: Comparison between MPU6050 and LSM6DS3TR-C for motion sensing

For the accelerometer the LSM6DS3TR-C has been selected. The LSM6DS3TR-C offers superior integration of gyroscope and accelerometer in a small, low-power package. Unlike the MPU6050, it includes a FIFO buffer and FSM features for onboard motion detection, reducing load on the MCU.

4.2.2 Barometric Pressure Sensor

Throughout the rocket’s flight, altitude is estimated using the barometer’s measurement of air pressure. It makes it possible to detect important flight events like descent and apogee, or the highest point and the activation of the parachute deployment depend on this data.

SPL05-001 Justification

The SPL06-001 is a high-resolution digital barometric pressure sensor ideal for measuring altitude changes during a rocket’s flight. It supports accurate apogee detection and surpasses the project’s $\pm 2\%$ altitude accuracy requirement with a resolution of 0.006 m and an accuracy of ± 2 Pa (± 0.17 m altitude error). The sensor can interface with the selected ESP32 MCU in a variety of ways thanks to its I²C or SPI communication capabilities. The SPL06-001 is ideal for lightweight and energy-constrained

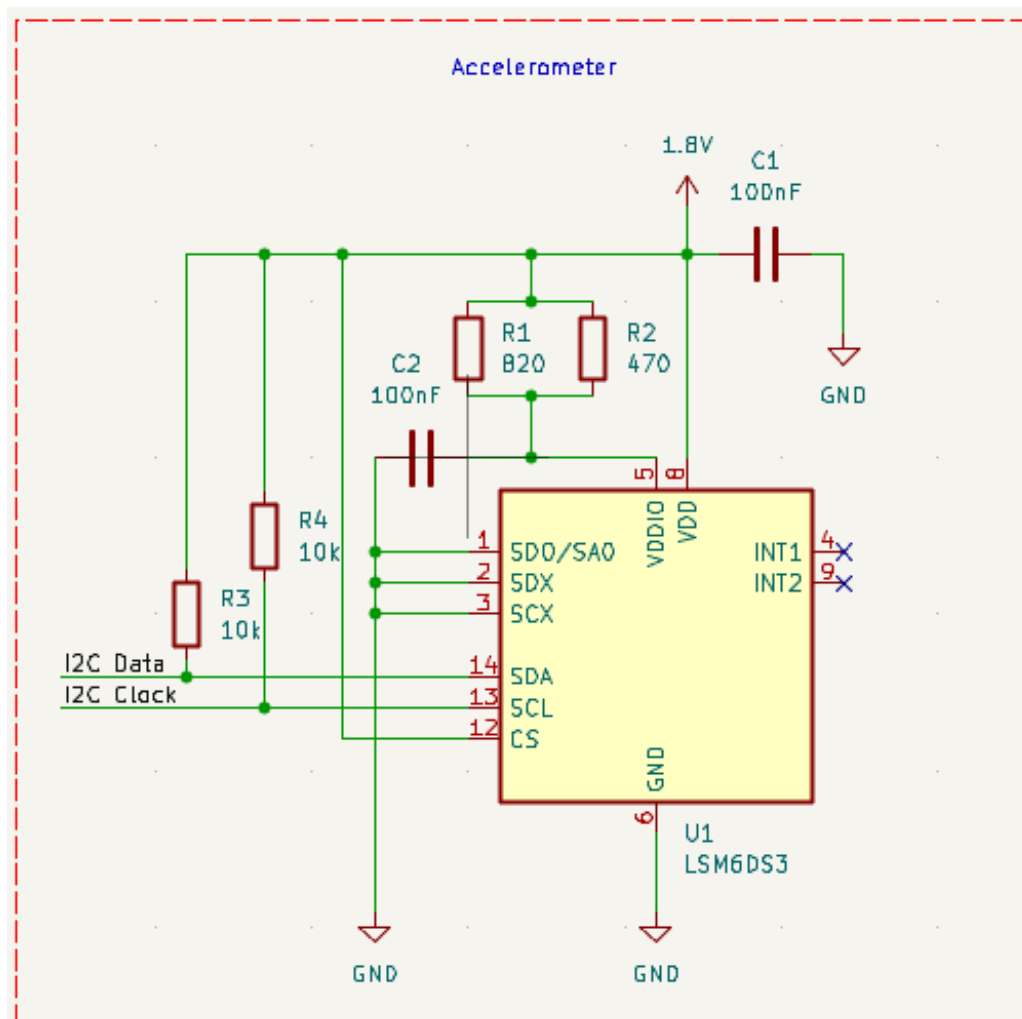


Figure 4.1: Accelerometer Schematic

systems due to its small form factor and low power consumption (down to 1 μA in standby), even with its high precision.

MS5611 Justification

The MS5611 is a high-resolution barometric pressure sensor well-suited for altitude estimation in model rocketry. It offers incredibly fine-grained altitude tracking that is appropriate for identifying minute changes in ascent and descent, with a pressure resolution of up to 0.012 mbar and a corresponding altitude resolution of roughly 0.01 m. Without the need for intricate external circuitry, the sensor can provide low-noise, high-accuracy pressure and temperature data thanks to its integrated 24-bit ADC and SPI or I²C interface.

Specification Adjusted Comparison

Specification ID	Specification	SPL06-001	MS5611
SP01	Target altitude (500m $\pm$ 50m)	$\pm$ 0.17m accuracy	$\pm$ 0.25m accuracy
SP02	Minimum 10Hz sampling rate	Up to 100Hz	50Hz (typical)
SP06	Apogee detection capability	High resolution and fast update	High resolution, moderate update
SP12	Altitude curve clarity	0.006m resolution	0.01m resolution
SP18	Power usage	1 μA standby / 7 μA avg	1 μA standby / 12 μA active
SP19	Altitude accuracy	$\pm$ 2 Pa ($\pm$ 0.17m)	$\pm$ 0.25m
–	Communication interface	I ² C / SPI	I ² C / SPI

Table 4.3: Comparison between SPL06-001 and MS5611 for altitude sensing

For the barometric pressure sensor the SPL06-001 was chosen. The SPL06-001 outperforms its competitors in both resolution and sampling speed. The MS5611 comes close in precision but lacks the SPL06's compactness and efficiency. Therefore, the SPL06-001 is optimal for reliable altitude tracking under rapid dynamic changes.

4.2.3 GPS Module

The GPS module provides position, altitude cross-reference, and ground-track data for the telemetry system, and enables recovery of the rocket after landing by transmitting live coordinates to the ground station. Two options were considered: the u-blox NEO-6M and the u-blox NEO-M8N.

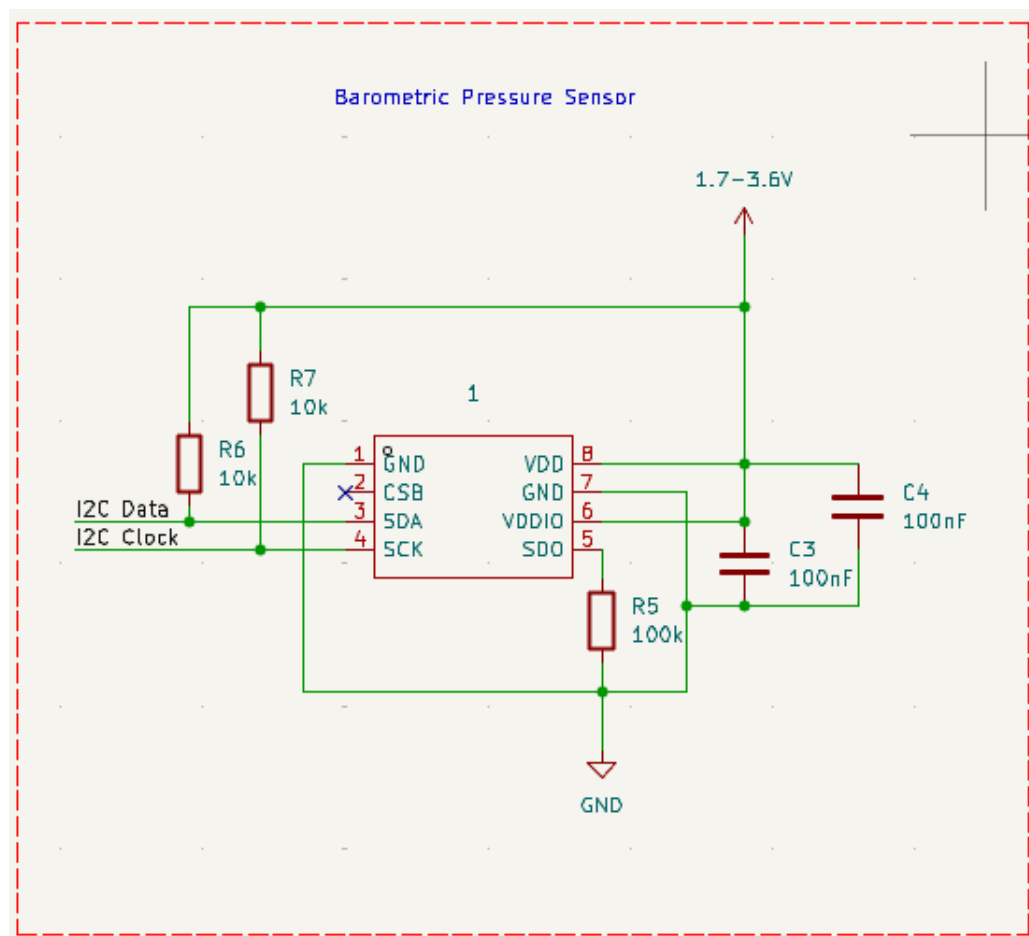


Figure 4.2: Barometer Schematic

NEO-6M Justification

The NEO-6M is a widely used u-blox GPS receiver module offering a good balance of cost, size, and performance for hobbyist rocketry applications. It provides a default update rate of 1Hz (configurable up to 5Hz), a horizontal position accuracy of approximately 2.5m, and a cold-start time-to-first-fix (TTFF) of around 27s, dropping to roughly 1s on hot start. Communication is handled over a simple UART interface, which integrates easily with the ESP32 and requires minimal additional wiring. Its onboard ceramic patch antenna and EEPROM for configuration storage make it a self-contained, low-complexity solution, and its low cost makes it well suited to a budget-conscious student project.

NEO-M8N Justification

The NEO-M8N is a more advanced u-blox GPS receiver capable of concurrent reception from multiple satellite constellations (GPS, GLONASS, Galileo, and BeiDou), which improves fix reliability in partially obstructed or fast-moving scenarios. It supports update rates up to 10Hz, achieves a tighter horizontal accuracy of around 2.0m, and offers improved receiver sensitivity (-167dBm vs -161dBm), leading to faster and more robust fixes during dynamic flight. It also communicates via UART (with I²C and SPI available), but its higher performance comes at increased cost and a larger form factor compared to the NEO-6M.

Specification Adjusted Comparison

Specification ID	Specification	NEO-6M	NEO-M8N
SP02	Minimum 10Hz sampling rate	Up to 5Hz	Up to 10Hz
SP01	Target altitude/position accuracy	2.5m CEP	2.0m CEP
SP13	Fix acquisition time (cold start)	27s	26s
SP20	Satellite constellation support	GPS only	GPS + GLONASS + Galileo + BeiDou
SP18	Power usage	45mA (acquisition)	35mA (acquisition)
SP21	Recovery/tracking reliability	Adequate, single constellation	Higher, multi-constellation redundancy
–	Communication interface	UART	UART / I ² C / SPI
–	Relative cost	Low	Moderate–High

Table 4.4: Comparison between NEO-6M and NEO-M8N GPS modules

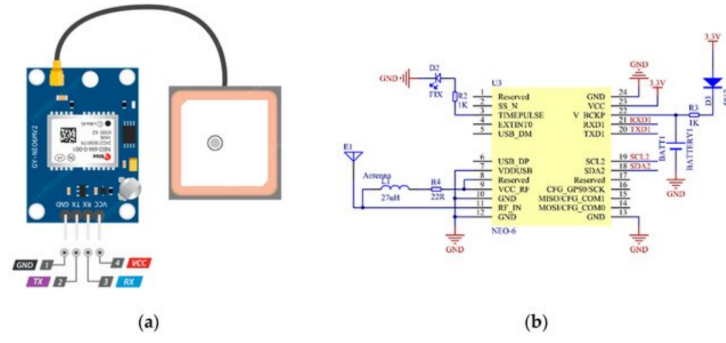


Figure 4.3: NEO-6M GPS Module Schematic

For the GPS module the NEO-6M was selected. Although the NEO-M8N offers superior sensitivity and multi-constellation support, the NEO-6M satisfies the project's positioning and recovery requirements at a significantly lower cost and complexity, making it the more practical choice given the budget and scope of a student rocketry project. Its 1–5Hz update rate is sufficient for post-launch recovery tracking, where high-frequency updates are less critical than during powered ascent (which is instead handled by the onboard IMU and barometer).

4.3 Power circuit

4.3.1 Voltage Regulation

Voltage regulation is crucial because it guarantees that all electronic components receive a precise and steady voltage, which is necessary for dependable system operation. Even slight changes in supply voltage can cause malfunctions, erroneous readings, or irreversible damage to the majority of sensors, microcontrollers, and communication modules.

LM317 Justification

The LM317 is a versatile adjustable linear voltage regulator that can output 1.25V to 37V with appropriate resistor configuration. Its robustness and simplicity make it widely used. The LM317 is appealing for general use and prototyping in model rocketry when power supply noise is not a major issue. Nevertheless, two external resistors are needed to adjust the output voltage, and large capacitors are frequently needed for stability.

MIC5219-3.3YC5-TR Justification

The MIC5219-3.3YC5-TR is a fixed-output 3.3V LDO regulator specifically designed for battery-powered embedded systems. It is perfect for controlling 3.3 V from a 3.7 V LiPo battery, even when under load, because of its extremely low dropout voltage of only 165 mV @ 500 mA. Additionally, it provides tight output tolerance ($\pm 1\%$) and low quiescent current (85 μ A), reducing power waste and guaranteeing stability.

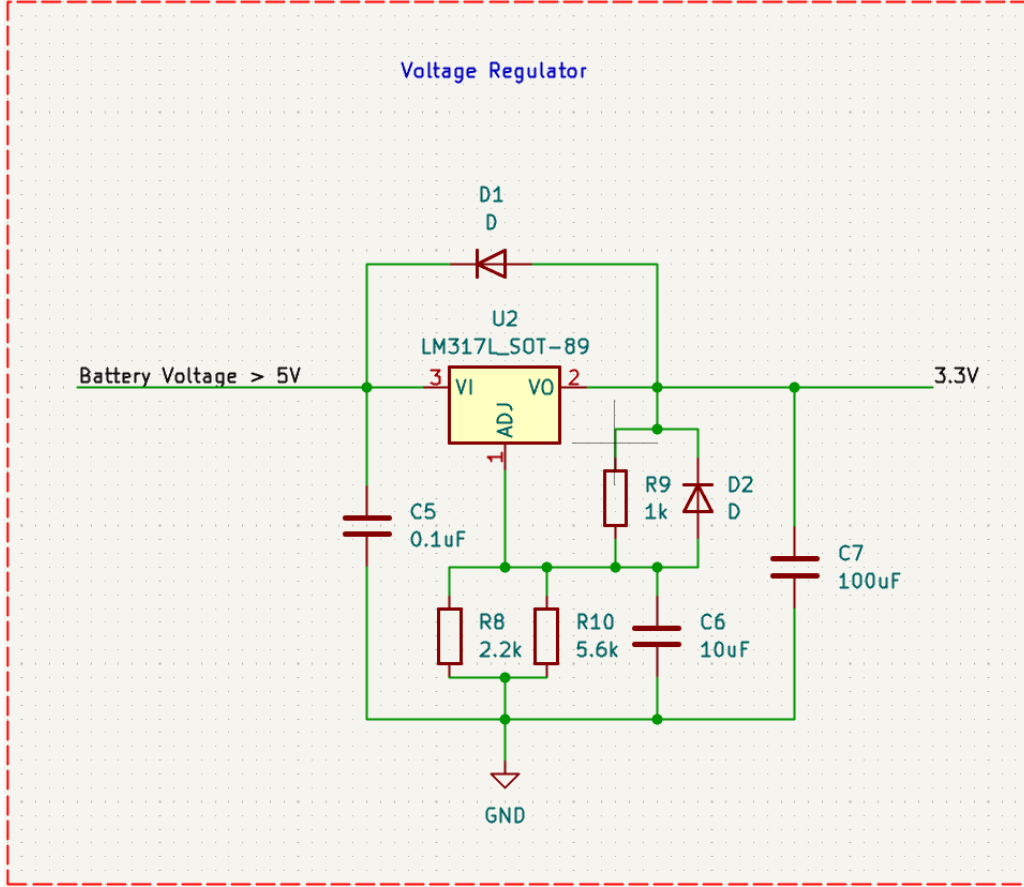


Figure 4.4: Enter Caption

Specification	LM317	MIC5219-3.3
Output Voltage	Adjustable	Fixed 3.3V
Dropout Voltage	2–3V	165mV @ 500mA
Output Current	Up to 1.5A	Up to 500mA
Quiescent Current	5mA	85 μ A
Package Size	TO-220 / TO-92 / SOT-223	SOT-23-5
Output Accuracy	$\pm 1.5\%$	$\pm 1.0\%$

Table 4.5: Comparison between LM317 and MIC5219-3.3 voltage regulators

For the voltage regulator the LM317 was chosen because of its dependability and adjustability, especially when the input voltage is high enough above the desired output.

4.3.2 Battery

The power system is supplied by a single-cell Lithium Polymer (LiPo) battery (3.7 V nominal, 4.2 V fully charged, 3.0 V safe cutoff, 850 mAh). This is a deliberately lightweight and compact choice for a small airframe, but it changes the voltage-regulation requirements from the original two-cell assumption: at 3.7–4.2 V input, the previously selected **LM317** (typical dropout $\approx 2\text{--}3\text{ V}$) cannot reliably hold a 3.3 V rail once the cell sags under load or discharges below $\approx 5.3\text{ V}$ equivalent headroom. Accordingly, the

MIC5219-3.3YC5-TR LDO (Table 4.4) is the correct regulator for this battery: with only 165 mV dropout at 500 mA, it maintains a stable 3.3 V rail down to a battery voltage of ≈ 3.47 V, covering nearly the full usable discharge curve of a single-cell pack.

A 2-hour continuous runtime (SP18) remains comfortably met, since the combined average draw of the ESP32, MPU6050, and BMP280 is under 150 mA, giving the 850 mAh cell a theoretical endurance in excess of 5 hours before accounting for the brief high-current igniter pulse (which is drawn directly from the battery, not through the LDO).

Battery Voltage Monitoring

Battery voltage is monitored using a resistive voltage divider feeding **GPIO34** on the ESP32. GPIO34 is used because it is an input-only ADC1 pin (ADC1_CH6) with no internal pull-up/pull-down resistors to load the divider, and it does not conflict with the WiFi radio’s shared circuitry on ADC2.

Because the single-cell pack’s full-charge voltage (4.2 V) is close to the ESP32 ADC’s maximum safe input (~ 3.3 V), the divider ratio must be more conservative than a 2S design would need:

$$V_{ADC} = V_{BAT} \times \frac{R_{bottom}}{R_{top} + R_{bottom}}$$

Using $R_{top} = 10 \text{ k}\Omega$ and $R_{bottom} = 22 \text{ k}\Omega$:

$$V_{ADC} = V_{BAT} \times \frac{22}{32} = V_{BAT} \times 0.6875$$

At full charge (4.2 V), $V_{ADC} \approx 2.89$ V — safely below the ADC ceiling with headroom for transients. At the 3.0 V safe discharge cutoff, $V_{ADC} \approx 2.06$ V. Firmware reconstructs the true battery voltage by dividing the raw ADC reading by 0.6875 (with a per-board calibration offset for ADC non-linearity) before publishing it on the `rocket/sensors` MQTT topic (`‘batt’` field, Listing 7.3). A firmware threshold of **3.4 V** — chosen with margin above the 3.0 V hard cutoff to protect the LiPo cell — triggers the `LOW_BATTERY` status shown on the ground station status bar (Figure 7.2).

Parameter	Specification
Battery chemistry	Single-cell (1S) LiPo
Nominal voltage	3.7 V
Full charge voltage	4.2 V
Safe discharge cutoff	3.0 V
Capacity	850 mAh
Voltage regulator	MIC5219-3.3YC5-TR (LDO, 165 mV dropout)
Monitoring pin	GPIO34 (ADC1_CH6, input-only)
Divider ratio	$R_{top} = 10 \text{ k}\Omega$, $R_{bottom} = 22 \text{ k}\Omega$ (0.6875)
Low-battery threshold	3.4 V

Table 4.6: Battery and voltage monitoring specifications (1S LiPo)

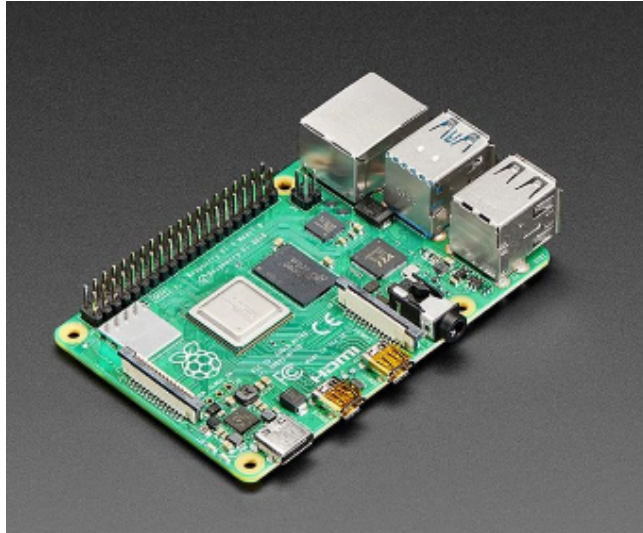


Figure 4.5: Raspberry Pi 3

4.4 Telemetry Computer

The telemetry computer collects real-time flight data (GPS position, altitude, acceleration, velocity, orientation, battery voltage) and transmits it wirelessly to the ground station. Unlike traditional LoRa-based systems, this design uses Raspberry Pi 3 with WiFi and MQTT protocol for high-bandwidth, low-latency telemetry transmission.

4.4.1 Telemetry Microcontorller

The Raspberry Pi 3 was selected over traditional microcontrollers (ESP32, STM32, NodeMCU) due to its superior processing power and built-in networking capabilities. It has a Quad-core 64-bit ARM Cortex-A53 @ 1GHz, 512MB LPDDR2 SDRAM and Built-in 2.4GHz WiFi 802.11n with Bluetooth 4.2

Justification

The Raspberry Pi 3 provides $5\times$ the processing power of ESP32 and includes native WiFi without additional modules. Running Linux (Raspberry Pi OS), it offers; Full Python ecosystem with extensive libraries (paho-mqtt, gpsd, smbus2). Real-time sensor fusion and data processing. Concurrent multi-sensor polling at high frequencies ($>100\text{Hz}$). Built-in file system for local data logging. SSH access for remote configuration and debugging. Systemd service integration for autonomous operation

Chapter 5

Ignition and Pyro System

5.1 Ignition System

Igniter

Nichrome Wire Justification

Despite being widely accessible and reasonably priced, nichrome ignition setups frequently need higher currents (1–3 A) and longer burn times to reliably ignite a motor, particularly in damp or windy conditions. Additionally, the ignition process is more susceptible to mechanical connections, which may lead to irregular firing. Additionally, nichrome components frequently need specialised mounting and insulation, which limits their suitability for plug-and-play or small-scale model rocketry designs.

Fuse Wire Justification

When current is applied, a fuse wire, a simple and affordable ignition method, burns through. Despite being easy to use, it has poor resistance to environmental factors like wind and humidity, inconsistent burn time, and low reliability. If it is inadvertently activated during handling, it may also be dangerous.

Electrical Match Justification

An electrical match, also known as an e-match, is a highly dependable, factory-sealed ignition device that starts combustion with a modest current, usually between 100 and 500 mA. It has a pyrotechnic compound that ignites instantly when triggered, guaranteeing quick, reliable ignition in a variety of environmental circumstances. Because e-matches are encapsulated for safety, there is a lower chance of unintentional ignition or environmental damage. They are particularly well-suited for low-voltage and battery-powered applications and integrate readily with microcontroller-based systems. E-matches are the recommended choice for model rocket ignition systems, especially when combined with automated launch electronics, because of their dependability, safety, and simplicity of use, which more than make up for their somewhat higher price.

5.1.1 MOSFETs

MOSFETs are vital in ignition systems due to their ability to switch high current loads quickly and efficiently with minimal power loss. When an external switch activates the MOSFET, it functions as a high-speed switch to supply current to an igniter in a model rocket ignition system. Low gate threshold voltage, low on-resistance $R_{DS(on)}$, and adequate current-handling capability are critical characteristics for ignition MOSFETs, particularly when powered by a 3.3V logic device such as an ESP32.

IRLZ44N Justification

The IRLZ44N is a logic-level N-channel MOSFET, with a maximum gate threshold voltage of 2V, meaning it can be fully turned on with a gate voltage as low as 3.3V. It can manage continuous drain current up to 47A, which is significantly more than required for a typical ignition system, and it has a very low $R_{DS(on)}$

IRF540N Justification

The IRF540N is a standard power MOSFET with outstanding switching characteristics that can handle high voltage and current. It usually needs a gate voltage of 10V or more to fully turn on, though, because it is not logic-level. Because of this, it is less appropriate for 3.3V logic systems unless a gate driver is added.

Specification	IRLZ44N	IRF540N
Gate Threshold Voltage	1.0V – 2.0V	2.0V – 4.0V
Logic Level Compatible	Yes	No
$R_{DS(on)}$ @ $V_{GS} = 10V$	0.022 Ω	0.077 Ω
Continuous Drain Current	47A @ 25°C	33A @ 25°C
Total Gate Charge	~67nC	~71nC
Package	TO-220	TO-220
Max V_{DS}	55V	100V

Table 5.1: Comparison between IRLZ44N and IRF540N MOSFETs

Because of its excellent current-handling capabilities, logic-level compatibility, and low gate threshold, the IRLZ44N was selected for the ignition system. There is no need for additional circuitry because microcontrollers like the ESP32 or mechanical switch can drive it directly using 3.3V logic. For small, dependable, and effective model rocket ignition systems, this makes it perfect.

5.2 Pyro System

5.2.1 Overview

The pyrotechnic (pyro) system is responsible for deploying the recovery parachute at the appropriate time during flight, typically at apogee. This system must be reliable, safe, and precisely timed to ensure controlled descent and safe recovery of the rocket. The pyro system consists of an ejection charge mechanism, parachute deployment hardware, and the parachute itself.

5.2.2 Ejection Charge Ignition System

Electric Match (E-match)

Justification: Electric matches (e-matches) are factory-sealed pyrotechnic igniters designed specifically for reliable, safe ignition of ejection charges in model rocketry. Unlike improvised igniters

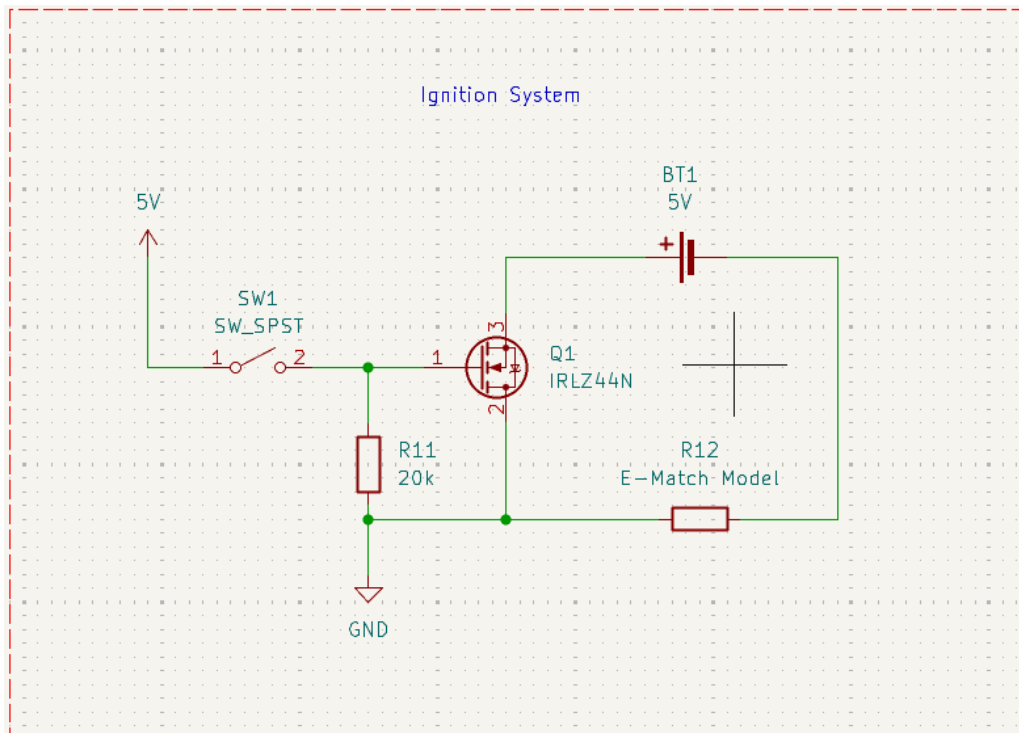


Figure 5.1: Ignition Circuit

(nichrome wire, fuse wire), e-matches provide consistent, instantaneous ignition when activated by the flight computer, making them the industry standard for parachute deployment systems.

How E-matches Work: An e-match consists of a small pyrotechnic compound (similar to match head composition) encapsulated in a protective shroud with two insulated wire leads. When current flows through the internal bridge wire (typically nichrome), it rapidly heats and ignites the pyrotechnic compound, producing a brief, intense flame that reliably ignites the black powder ejection charge.

Key Advantages:

- **Low current requirement:** Ignites reliably with 100–500mA (vs 1–3A for bare nichrome wire)
- **Instantaneous ignition:** <10ms activation time ensures precise deployment timing
- **Factory-sealed construction:** Resistant to moisture, humidity, and environmental contamination
- **Consistent performance:** Minimal variation in ignition delay or energy output
- **Safety:** Encapsulated design prevents accidental ignition during handling
- **Reliability:** >99% ignition success rate in field conditions
- **Temperature tolerance:** Operates reliably from -40°C to $+80^{\circ}\text{C}$

Technical Specifications (Typical):

Parameter	Specification
Ignition Current	100–500mA (depending on model)
No-Fire Current	<50mA (safe handling threshold)
All-Fire Current	1A (guaranteed ignition)
Bridge Wire Resistance	1–2 Ω (varies by manufacturer)
Response Time	<10ms at rated current
Ignition Energy	~20–100mJ
Wire Length	15–45cm (customizable)
Wire Gauge	20–26 AWG
Shroud Material	Plastic or paper tube

Table 5.2: E-match technical specifications

5.2.3 Parachute Configuration

Hemispherical (Dome) Parachute

The hemispherical parachute uses a circular canopy of 6–12 curved gore panels with a spill hole at the apex (5–10% of diameter) and shroud lines of $1.5\times$ the canopy diameter. It offers a predictable drag coefficient ($C_d \approx 0.75\text{--}0.8$) and stable, low-oscillation descent with well-documented performance characteristics. However, the curved gore panels require precision cutting and sewing, making construction more complex and time-consuming than flat-pattern alternatives.

Cruciform (X-form) Parachute

The cruciform parachute consists of four rectangular panels arranged in a “+” pattern, producing a higher drag coefficient ($C_d \approx 1.4\text{--}1.5$) with no spill hole required. Its straight seams and four-piece construction make it the easiest to fabricate, with fast deployment and a compact flat-fold packed volume. The trade-off is increased pendulum oscillation during descent, a less predictable descent rate, and higher horizontal wind drift compared to symmetric canopy designs.

Octagonal Flat Parachute

The octagonal flat parachute is cut from a single sheet of fabric into an eight-sided polygon, with a central spill hole (10–15% of diameter) and eight shroud lines attached at the vertices. This design strikes the best balance among the three options: its $C_d \approx 0.85\text{--}0.95$ lies between the hemispherical and cruciform values, its eight symmetric attachment points produce a stable, predictable descent, and the single-piece flat pattern requires no curved seams, making it the simplest to construct and the most material-efficient. The only notable drawbacks are that it requires a slightly larger diameter than a cruciform to achieve the same descent rate, and it is less common in commercial rocketry.



Figure 5.2: Hemispherical parachute



Figure 5.3: Cruciform parachute

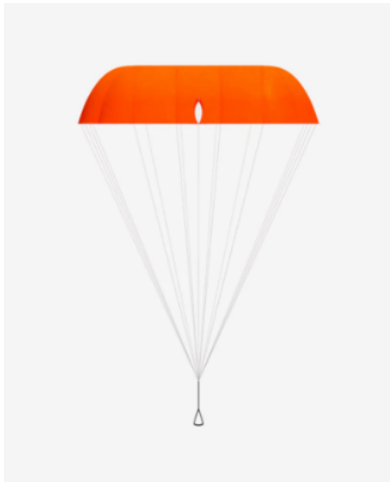


Figure 5.4: Octagonal flat parachute

Configuration	C_d	Stability	Construction	Selection
Hemispherical	0.75–0.80	Excellent	Complex (curved gores)	–
Cruciform	1.40–1.50	Moderate	Simple (straight seams)	–
Octagonal flat	0.85–0.95	Good	Simplest (single sheet)	✓

Table 5.3: Parachute configuration comparison

The **octagonal flat parachute (75 cm diameter, 31 cm side length)** was selected for its optimal balance of stability, predictable performance, and construction simplicity. This configuration meets SP07 safe-descent requirements while being well-suited to educational fabrication and repeated use.

5.3 Motor Selection Overview

Model rocket motors are classified by total impulse into power classes ranging from A through O and beyond. For this project the design constraint limits the selection to Class A–E (low-to-mid power). Two candidate motors were evaluated: the **Estes C6-5** and the **Estes D12-5**.

Motor Classification Background

Each alphanumeric motor designation encodes three pieces of information:

- **Letter (power class)** — Total impulse. Each successive letter doubles the impulse: A = 1.26 Ns, B = 2.5 Ns, C = 5.0 Ns, D = 10 Ns, E = 20 Ns.
- **First number (average thrust)** — Average thrust in Newtons. For the C6-5, this is 6 N.
- **Second number (delay)** — Time in seconds between motor burn-out and ejection-charge firing. For the C6-5 this is 5 s, timed to coincide with apogee on a typical low-mass airframe.

Candidate Motors & Comparative Analysis

Two commercially available Estes motors were considered: the **C6-5** and **D12-5**. The D12-5 provides approximately 70% greater total impulse and higher thrust, but requires a larger 24 mm motor mount and increases vehicle mass. The C6-5 is lighter, compatible with the existing 18 mm design, and provides sufficient performance for the mission requirements. A comparison of the motors is given in Table 5.4.

Table 5.4: Motor comparison against project specifications

Specification	Requirement	C6-5	D12-5
SP01 — Target altitude	500 m $\pm$ 50 m	$\approx$ 200–300 m (typical 200 g airframe)	$\approx$ 350–500 m (typical 300 g airframe)
SP07 — Max decel on landing	$\leq$ 10 G	Lower descent speed; gentler impact	Higher kinetic energy; borderline
SP08 — Non-metallic airframe	Required	18 mm mount; lighter structure adequate	24 mm mount; heavier structure needed
SP09 — Launch accel tolerance	Up to 15 G	Peak $\approx$ 7 G on 200 g rocket	Peak $\approx$ 10 G on 300 g rocket
SP16 — Basic-tools construction	Required	18 mm commonplace; easy to source	24 mm less common locally

OpenRocket Simulation Results

The C6-5 was simulated in OpenRocket with the as-designed airframe (total lift-off mass $\approx$ 200 g, diameter 40.4 mm, length 338.9 mm, four trapezoidal fins). Key simulation outputs are shown in Table 5.5.

Table 5.5: OpenRocket simulation summary — C6-5 motor

Parameter	Simulated Value
Predicted apogee	$\approx$ 219 m AGL
Max velocity	102 m/s (Mach 0.30)
Max acceleration	$\approx$ 68 m/s ² ($\approx$ 6.9 G)
Stability margin (C6-5)	2.44 cal / 15.2%
CG (no motor)	27.3 cm from nose
CP	34 cm from nose

The stability margin of 2.44 calibres is within the recommended range of 1–2 calibres and confirms stable, non-oscillatory flight.

Selected Motor: Estes C6-5

The **Estes C6-5** was selected due to its compatibility with the rocket design, electronics, and project requirements. The 18 mm motor fits the existing airframe, while the 5 s delay complements the ESP32 apogee-detection system and provides a backup deployment mechanism. The predicted apogee of approximately 219 m is suitable for initial testing, with scope for future optimisation. Peak acceleration

and landing loads remain within safety requirements, and the motor is widely available and cost-effective in South Africa.

Chapter 6

CAD Design

6.1 Overview

All structural components were modelled in CAD and verified using OpenRocket simulation prior to fabrication. The rocket comprises five primary structural assemblies: the nose cone, body tube, electronics bay, fin assembly, and motor mount. A dedicated launch pad with guide rod and base structure was also designed to support repeatable, safe launch operations.

6.1.1 Full Assembly

Figure 6.1 shows the complete rocket assembly integrating all subsystems. The overall configuration follows a conventional single-stage layout with the motor at the aft, fins providing aerodynamic stability, the electronics bay mid-body, and the nose cone housing the parachute bay at the fore.

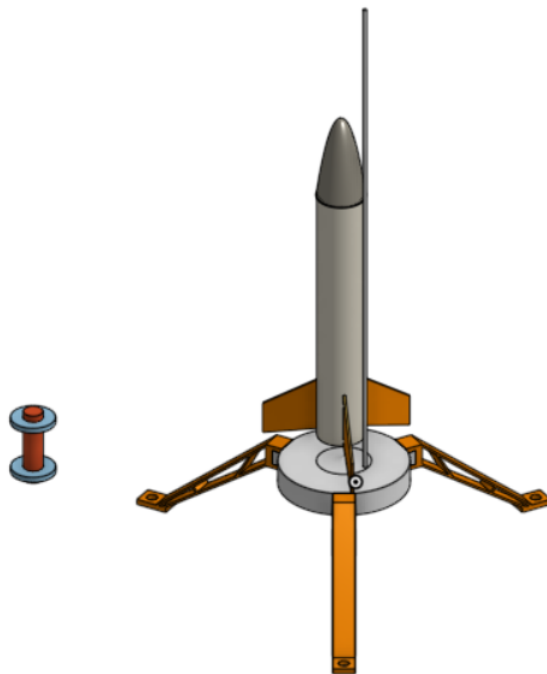


Figure 6.1: Full rocket 3D assembly

6.1.2 OpenRocket Simulation

The rocket geometry and mass distribution were validated in OpenRocket (Figure 6.2) to confirm aerodynamic stability, projected apogee, and centre-of-pressure margin before committing to fabrication dimensions.

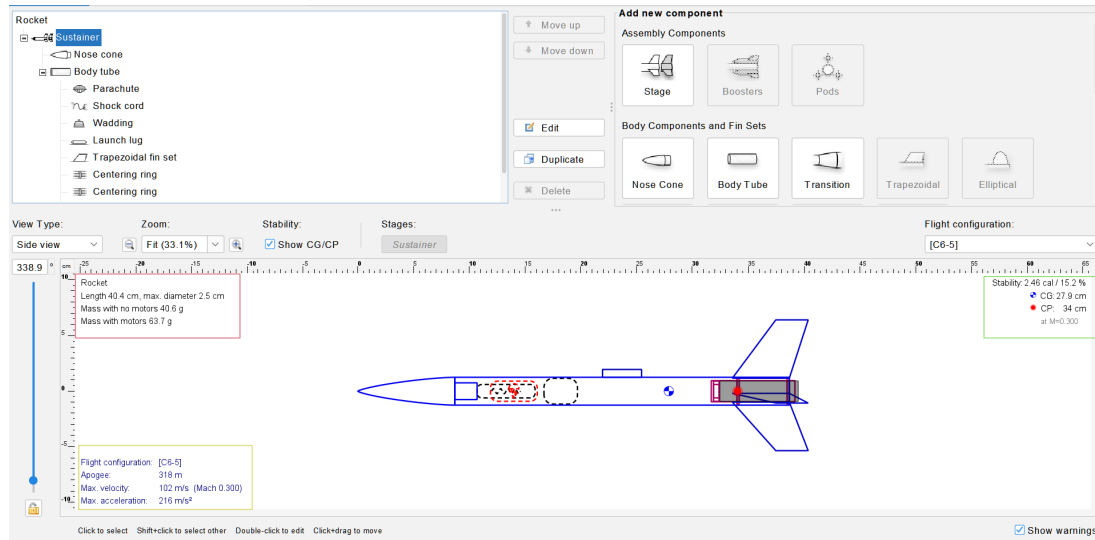


Figure 6.2: OpenRocket simulation showing stability margin and flight profile

6.1.3 Nose Cone

The nose cone was designed with an ogive profile to minimise drag at subsonic velocities. It is split into front and bottom sections to allow the parachute bay to be accessed for packing and recovery.

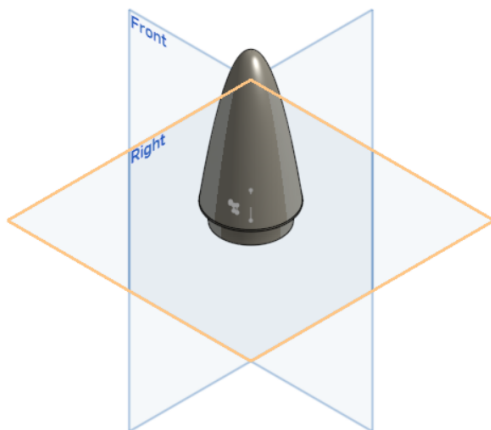


Figure 6.3: Nose cone 3D view

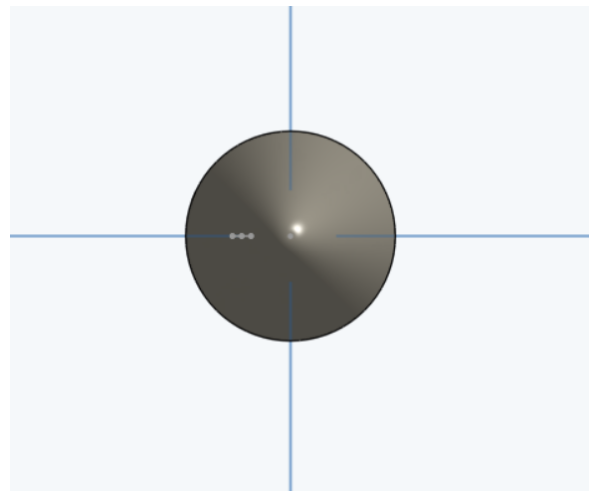


Figure 6.4: Nose cone front profile

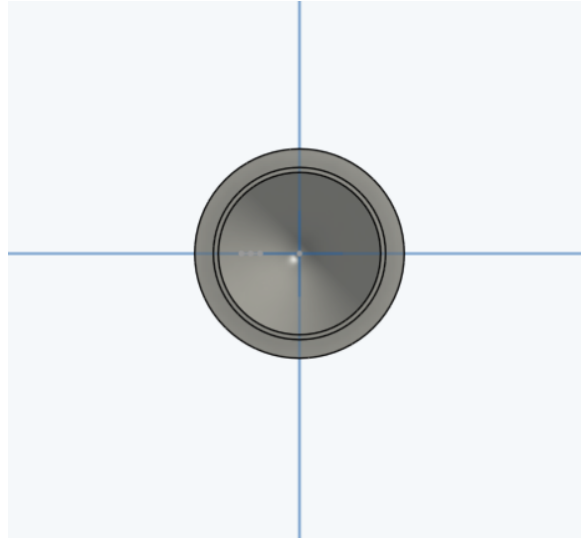


Figure 6.5: Nose cone base detail showing parachute bay interface

6.1.4 Body Tube

The body tube forms the main structural airframe, connecting the nose cone to the fin and motor assembly. Its internal diameter is sized to accommodate the electronics bay and parachute volume, while wall thickness was selected to provide adequate structural stiffness under motor thrust loads.

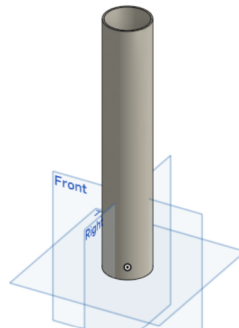


Figure 6.6: Body tube dimensioned drawing

6.1.5 Electronics Bay

The electronics bay is a self-contained removable section that houses the flight computer (ESP32), barometric pressure sensor, IMU, battery, and ignition circuit. The bay is accessible from the exterior to allow pre-launch arming and post-flight data retrieval.

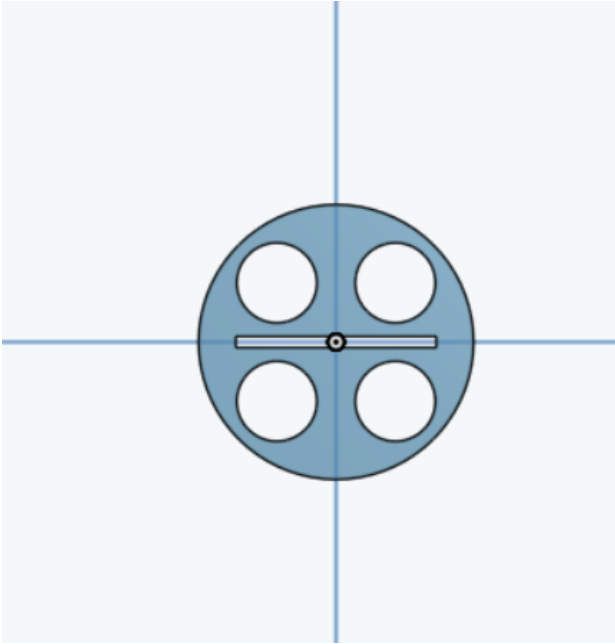


Figure 6.7: Electronics bay dimensioned drawing

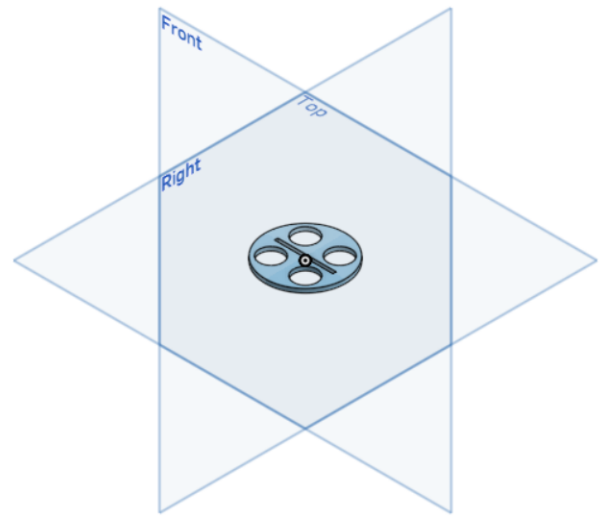


Figure 6.8: Electronics bay 3D view

6.1.6 Fin Assembly

Four trapezoidal fins are mounted symmetrically at the aft of the body tube to provide aerodynamic stability. The fin geometry and sweep angle were determined from the OpenRocket stability analysis to achieve a positive stability margin of at least 1 calibre.

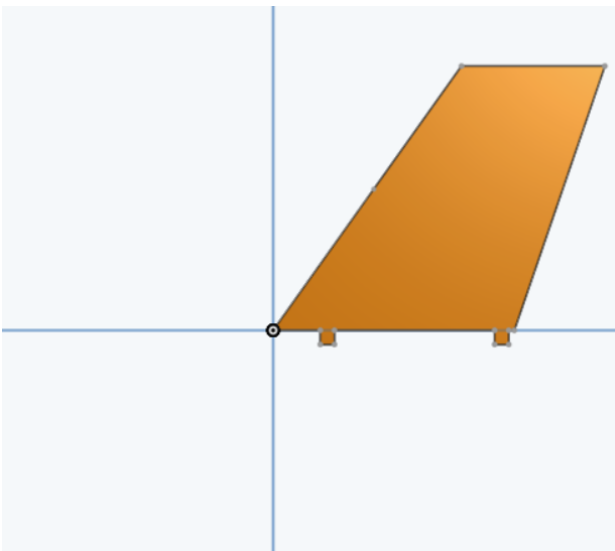


Figure 6.9: Individual fin dimensioned drawing

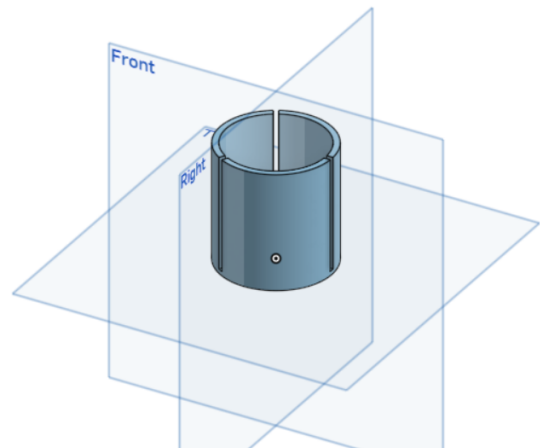


Figure 6.10: Fin alignment jig drawing

6.1.7 Motor Mount

The motor mount retains the rocket motor within the aft airframe and transfers thrust to the body tube structure. It is designed to accept a standard 29 mm motor casing with a positive retention clip to prevent ejection under motor burn-out pressure.

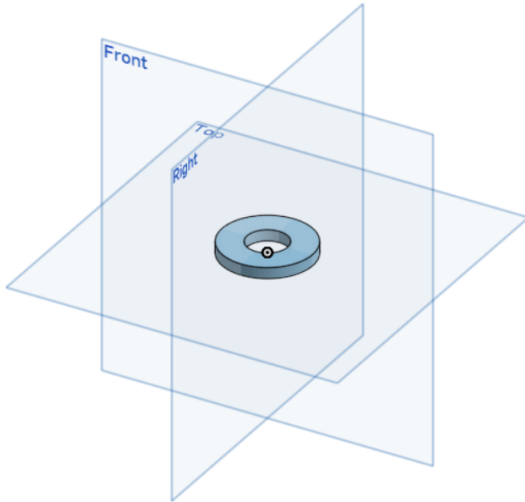


Figure 6.11: Motor mount dimensioned drawing

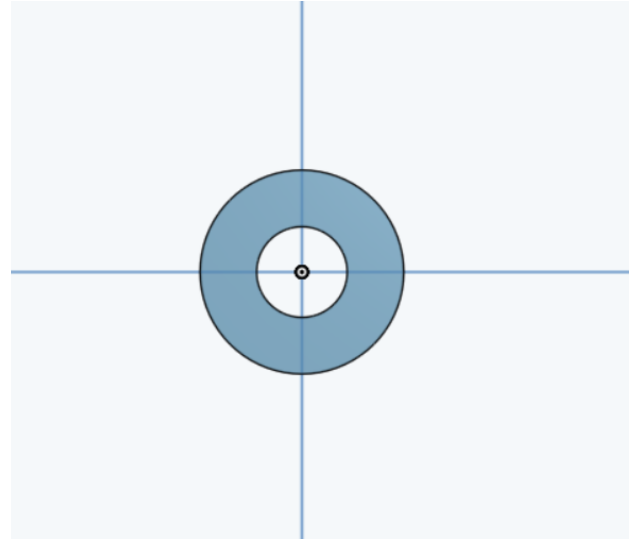


Figure 6.12: Motor mount 3D view

6.1.8 Launch Lug

The launch lug slides over the launch rod, constraining the rocket to a vertical trajectory until sufficient velocity is reached for aerodynamic stability. The lug is designed with a smooth bore to minimise friction and is positioned to avoid interference with the fin assembly.

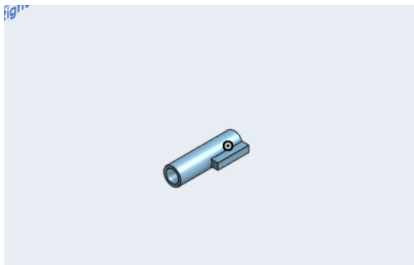
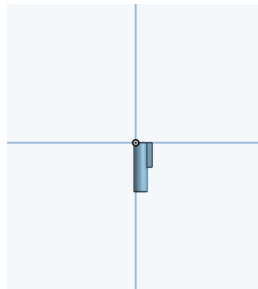
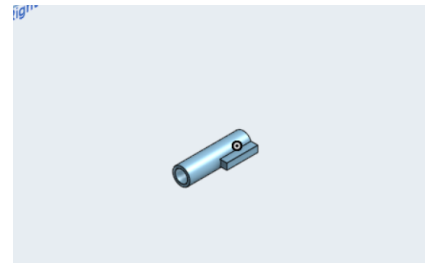


Figure 6.13: Launch lug 3D view

Figure 6.14: Launch
lug side viewFigure 6.15: Launch lug top
view

6.1.9 Launch Pad Assembly

A dedicated launch pad was designed to support the rocket on the launch rod at the correct angle. The system comprises a tripod base, angled pad base, and a vertical guide rod. The base provides sufficient footprint to remain stable on uneven ground during launch.

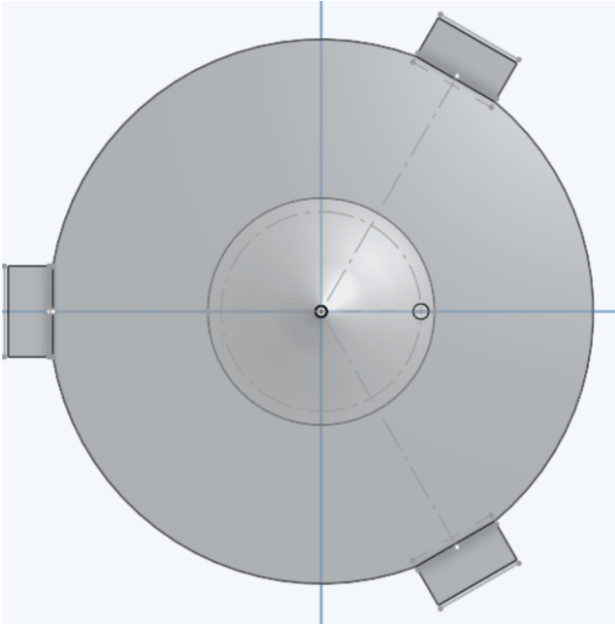


Figure 6.16: Pad base dimensioned drawing

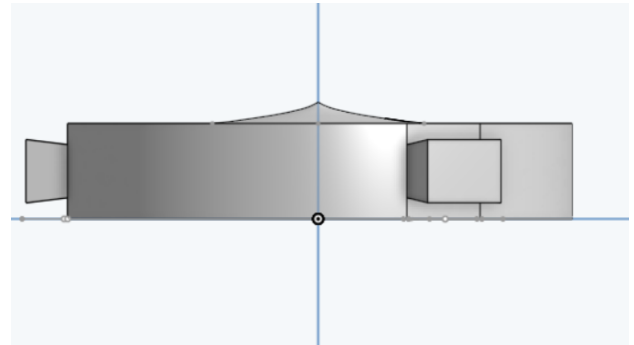


Figure 6.17: Pad base side view

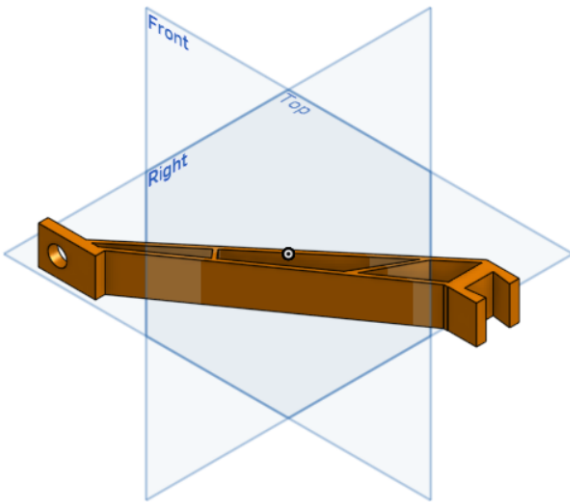


Figure 6.18: Pad leg 3D view

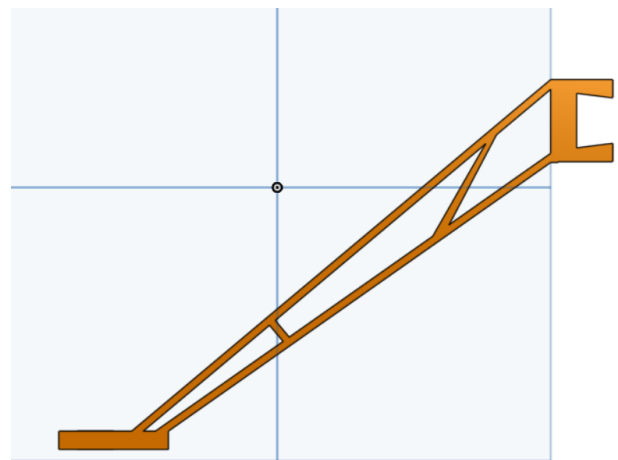


Figure 6.19: Pad leg side view

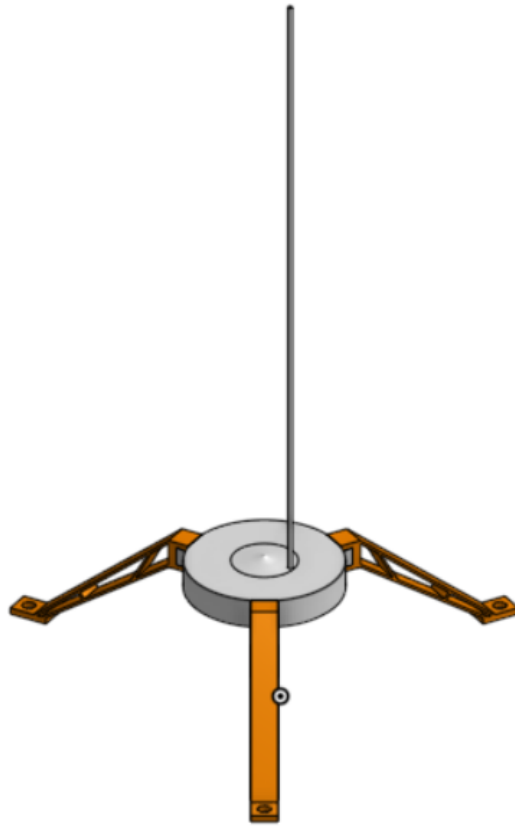


Figure 6.20: Full launch pad assembly with rocket mounted on rod

Chapter 7

Ground Station Design

7.1 Ground Station Architecture Overview

The ground station is built around a **Raspberry Pi 4** (Pi 4B, 4 GB RAM) running a Python/Flask web application. The architecture is shown in Figure 7.1 and comprises four layers:

1. **Transport layer** — 2.4 GHz WiFi (802.11n) between the ESP32 on the rocket and the Pi 4 acting as both MQTT broker and WiFi access point.
2. **Message broker** — Mosquitto MQTT broker running on the Pi 4, subscribing to all `rocket/#` topics published by the ESP32.
3. **Application layer** — Flask web server with Flask-SocketIO providing real-time WebSocket push to any connected browser.
4. **Presentation layer** — Responsive HTML5 dashboard (`index.html`) rendered in a browser on any device on the local network, displaying live charts, GPS map, and flight event log.

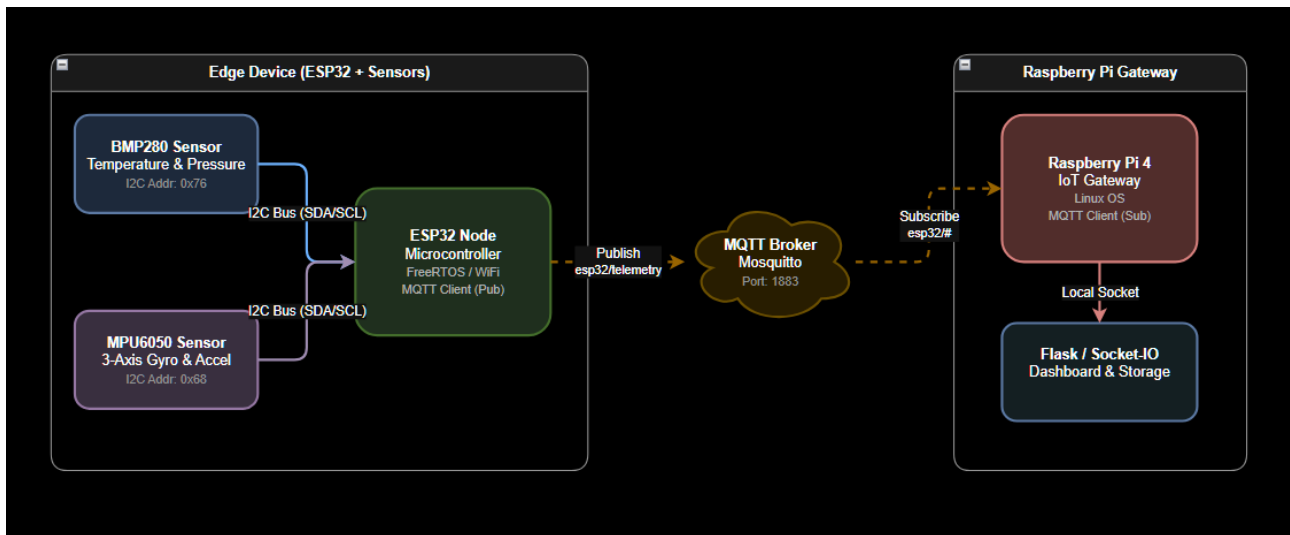


Figure 7.1: Ground station system architecture

7.2 Ground Station Hardware

7.2.1 Microcontroller / Computer Selection: Raspberry Pi 4

The Raspberry Pi 4 Model B was selected as the ground station computer. Its key specifications are given in Table 7.1.

Table 7.1: Raspberry Pi 4 Model B specifications

Parameter	Specification
Processor	Quad-core Cortex-A72 @ 1.8 GHz (64-bit ARM)
RAM	4 GB LPDDR4-3200
WiFi	2.4 GHz / 5.0 GHz 802.11ac
Bluetooth	5.0 / BLE
USB	2× USB 3.0, 2× USB 2.0
Video	2× micro-HDMI (up to 4K)
Storage	microSD (OS) + external USB SSD (logs)
Operating System	Raspberry Pi OS (Debian Bookworm, 64-bit)
Power	5 V / 3 A via USB-C

Justification

The Pi 4 was chosen over the Pi 3 (used in the original telemetry computer plan) and over a laptop-based solution for the following reasons:

- **Processing power:** The Cortex-A72 cores run the Mosquitto MQTT broker, Flask web server, SocketIO event loop, and CSV logger concurrently without performance degradation at 10 Hz telemetry rates.
- **Dual-band WiFi:** The 5 GHz radio can serve browser clients while the 2.4 GHz radio maintains the link to the ESP32 rocket (which only has 2.4 GHz), avoiding channel interference.
- **Portability:** The Pi 4 runs from a USB-C power bank, making the ground station fully portable for field launches.
- **Full Linux ecosystem:** Systemd service management enables auto-start of the ground station application on boot, requiring no operator interaction beyond powering on the unit.
- **Cost:** Significantly less expensive than a laptop-based solution while providing more than adequate compute resources for this application.

7.3 Software Stack

7.3.1 Dependencies

The Python dependencies are managed in a virtual environment and listed in `requirements.txt`:

```
Flask==3.0.0
flask-socketio==5.3.5
paho-mqtt==1.6.1
python-socketio==5.10.0
eventlet==0.33.3
```

Listing 7.1: requirements.txt

Additionally, the **Mosquitto MQTT Broker** is installed at the OS level:

```
sudo apt-get update
sudo apt-get install -y mosquitto mosquitto-clients
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
```

Listing 7.2: System-level dependency installation

7.3.2 MQTT Topic Architecture

The ESP32 firmware publishes telemetry as compact JSON payloads on two primary topics. The ground station subscribes to the wildcard `rocket/#` pattern, catching all subtopics simultaneously.

Table 7.2: MQTT topic map — ESP32 to Ground Station

Topic	Payload Type	Description
rocket/sensors	JSON object	Kalman altitude, baro alt, velocity, temp, pressure, 3-axis accel, 3-axis gyro, battery voltage
rocket/gps	JSON object	Latitude, longitude, GPS altitude, satellites, speed
rocket/status	String	ONLINE / LAUNCHED / FLIGHT / LOW_BATTERY
rocket/wifi/rssi	String	WiFi RSSI in dBm

The JSON structure published on `rocket/sensors` by the ESP32 is:

```
1 {
2   "alt": kAlt,      // Kalman-filtered altitude (m, relative to ground)
3   "baro": baroAlt,  // Raw barometric altitude (m, relative to ground)
4   "vel": kVel,      // Kalman-filtered vertical velocity (m/s)
5   "temp": temperature, // BMP280 temperature (deg C)
6   "press": pressure, // Atmospheric pressure (hPa)
7   "ax": accel.x,    // X-axis acceleration (m/s^2)
8   "ay": accel.y,    // Y-axis acceleration (m/s^2)
9   "az": accel.z,    // Z-axis acceleration (m/s^2)
10  "gx": gyro.x,     // X-axis angular rate (rad/s)
11  "gy": gyro.y,     // Y-axis angular rate (rad/s)
12  "gz": gyro.z,     // Z-axis angular rate (rad/s)
13  "batt": battVoltage // Battery voltage (V)
14 }
```

Listing 7.3: ESP32 sensor JSON payload (from firmware)

7.3.3 Main Application: ground_station_app.py

The main application is a Flask server with SocketIO integration. The key functional blocks are described below.

MQTT Client Initialisation

```

1 import paho.mqtt.client as mqtt
2
3 MQTT_BROKER = "localhost" # Mosquitto running on same Pi 4
4 MQTT_PORT = 1883
5 MQTT_TOPIC_PREFIX = "rocket"
6
7 mqtt_client = mqtt.Client("ground_station_web")
8 mqtt_client.on_connect = on_connect
9 mqtt_client.on_message = on_message
10 mqtt_client.on_disconnect = on_disconnect
11
12 mqtt_client.connect(MQTT_BROKER, MQTT_PORT, 60)
13 mqtt_client.loop_start()

```

Listing 7.4: MQTT client setup and connection

Connection and Subscription Callback

On a successful MQTT connection the client subscribes to all rocket topics using the wildcard `rocket/#`:

```

1 def on_connect(client, userdata, flags, rc):
2     print(f"Connected to MQTT broker with result code {rc}")
3     client.subscribe(f"{MQTT_TOPIC_PREFIX}/#")
4     telemetry_data['status'] = 'CONNECTED'
5     socketio.emit('status_update', {'status': 'MQTT Connected'})

```

Listing 7.5: on_connect callback

Message Processing and Real-Time Push

The `on_message` callback decodes incoming MQTT payloads, updates the in-memory telemetry store, appends to the rolling history deque (500-point window), updates flight statistics, and immediately pushes the updated state to all connected browser clients via SocketIO:

```

1 def on_message(client, userdata, msg):
2     topic = msg.topic
3     payload = msg.payload.decode('utf-8')
4
5     # Status string
6     if topic == f"{MQTT_TOPIC_PREFIX}/status":
7         telemetry_data['status'] = payload
8         socketio.emit('status_update', {'status': payload})
9

```

```

10 # Combined sensor JSON from ESP32
11 elif topic == f"{MQTT_TOPIC_PREFIX}/sensors":
12     data = json.loads(payload)
13     alt = data.get('alt', 0)
14     vel = data.get('vel', 0)
15
16     # Update telemetry store
17     telemetry_data['sensors']['altitude_baro'] = data.get('baro', 0)
18     telemetry_data['sensors']['accel'] = {
19         'x': data.get('ax', 0),
20         'y': data.get('ay', 0),
21         'z': data.get('az', 0)
22     }
23     telemetry_data['battery'] = data.get('batt', 0)
24
25     # Append to history (500-point rolling window)
26     current_time = time.time()
27     history_data['altitude'].append(alt)
28     history_data['velocity'].append(vel)
29     history_data['timestamps'].append(current_time)
30
31     # Update flight statistics
32     if alt > flight_stats['max_altitude']:
33         flight_stats['max_altitude'] = alt
34
35 # GPS JSON
36 elif topic == f"{MQTT_TOPIC_PREFIX}/gps":
37     gps = json.loads(payload)
38     telemetry_data['gps'].update({
39         'lat': gps.get('lat', 0),
40         'lon': gps.get('lon', 0),
41         'alt': gps.get('alt', 0),
42         'satellites': gps.get('sat', 0),
43         'speed': gps.get('spd', 0)
44     })
45
46 # Launch detection via status string
47 if payload == "LAUNCHED":
48     flight_stats['launch_time'] = time.time()
49     flight_stats['current_state'] = 'ASCENT'
50     flight_events.append({'type': 'LAUNCH', 'timestamp': time.time()})
51     socketio.emit('flight_event', {'type': 'LAUNCH'})
52
53 # Battery warning
54 if topic == f"{MQTT_TOPIC_PREFIX}/status" and payload == "LOW_BATTERY":
55     socketio.emit('flight_event', {'type': 'LOW_BATTERY'})
56
57 # Timestamp and push to all browser clients
58 telemetry_data['timestamp'] = time.time()
59 socketio.emit('telemetry_update', telemetry_data)
60
61 # Log to CSV if active
62 if logging_active and csv_writer:

```



```
63 log_telemetry_data()
```

Listing 7.6: Simplified on_message callback (key logic)

Background Update Task

A SocketIO background task runs at 10 Hz to ensure clients receive regular updates even during periods of reduced MQTT traffic:

```
1 def background_updates():
2     while True:
3         socketio.sleep(0.1)           # 10 Hz
4         socketio.emit('telemetry_update', telemetry_data)
5
6         # Derive velocity from altitude history when Kalman vel not available
7         if len(history_data['altitude']) >= 2:
8             dt = (history_data['timestamps'][-1]
9                  - history_data['timestamps'][-2])
10            if dt > 0:
11                dh = (history_data['altitude'][-1]
12                     - history_data['altitude'][-2])
13                velocity = dh / dt
14                history_data['velocity'].append(velocity)
15                if abs(velocity) > flight_stats['max_velocity']:
16                    flight_stats['max_velocity'] = abs(velocity)
17
18 socketio.start_background_task(background_updates)
```

Listing 7.7: Background 10 Hz update task

7.3.4 Flask REST API Endpoints

In addition to the SocketIO real-time channel, a set of REST endpoints allows any HTTP client to query or control the ground station:

Table 7.3: Flask REST API endpoints

Endpoint	Method	Description
/api/telemetry	GET	Returns current telemetry snapshot as JSON
/api/history	GET	Returns 500-point history arrays (altitude, acceleration, timestamps)
/api/events	GET	Returns chronological flight event log
/api/stats	GET	Returns flight statistics (max alt, vel, accel, flight time)
/api/command	POST	Sends ARM / DISARM / ABORT / DEPLOY command to rocket via MQTT
/api/logging/start	POST	Begins CSV telemetry logging to /home/pi/ground_station_logs/
/api/logging/stop	POST	Stops CSV logging
/api/reset	POST	Clears flight stats, history, and event log

7.3.5 Data Logging

Logging state is tracked by three module-level globals: `log_file`, which holds the open file handle; `csv_writer`, a Python `csv.writer` instance bound to that handle; and the boolean flag `logging_active`. All three are initialised to `None` / `False` at startup, so no I/O occurs until logging is explicitly enabled.

Logging is controlled at runtime via two REST endpoints. A `POST` to `/api/logging/start` calls `start_logging()`, which captures the current wall-clock time via `datetime.now()`, constructs the output path, opens the file with `newline=""` as required by Python's `csv` module, writes a fixed header row containing all sixteen column names, and sets `logging_active = True`. A subsequent `POST` to `/api/logging/stop` calls `stop_logging()`, which flushes and closes the file handle and resets all three globals to their idle state.

Each session produces a single file named `flight_YYYYMMDD_HHMMSS.csv`, where the timestamp is taken at the moment `start_logging()` is called, ensuring filenames are unique across sessions even if multiple flights occur on the same day.

Files are stored under `~/rocket_ground_station/ground_station_logs/`; the directory is created automatically via `os.makedirs(exist_ok=True)` on first use, so no manual setup is required after installation.

Rows are written inside the MQTT message pipeline. The broker publishes telemetry on a subscribed topic; each arriving packet fires the `on_message` callback, which runs on the background thread started by `mqtt_client.loop_start()`. After parsing and updating the in-memory telemetry store, the callback checks both `logging_active` and `csv_writer` before invoking `log_telemetry_data()`, which appends a single row to the open writer. The full column schema is shown in Table ??.

Column	Unit / Description
timestamp	Unix epoch (s)
gps_lat	Degrees (WGS-84)
gps_lon	Degrees (WGS-84)
gps_alt	Metres MSL
gps_speed	m/s
satellites	Count
altitude_baro	Metres AGL (Kalman-filtered)
accel_x/y/z	m/s ²
gyro_x/y/z	rad/s
temperature	°C
battery	Volts
rsi	dBm

Table 7.4: CSV log file column schema

7.3.6 Systemd Auto-Start Service

The ground station is configured to start automatically on Pi 4 boot via a systemd service unit. Table 7.5 summarises the key directives in `ground-station.service` in place of the full unit-file listing.

Table 7.5: Key directives in the `ground-station.service` systemd unit

Directive	Effect
After / Wants	Starts only once networking and the Mosquitto service are available
Type=simple	Runs as a standard foreground service process
WorkingDirectory	/home/pi/rocket_ground_station
ExecStart	Launches <code>ground_station_app.py</code> from the project's virtual environment
Restart=on-failure	Automatically restarts the service if it crashes
RestartSec=10	Waits 10s before each restart attempt
WantedBy=multi-user.target	Enables the service to start automatically at boot

This ensures the ground station web interface is available as soon as the Pi 4 connects to power, with no operator interaction required beyond booting the device.

7.4 Web Dashboard Interface

The dashboard (`templates/index.html`) is served by Flask and rendered in any modern browser on the local network at `http://<pi4-ip>:5000`. It is structured into the following panels.

7.4.1 Status Bar

The top bar provides at-a-glance system health indicators:

- **MQTT Status** — Green when the Flask app is connected to the Mosquitto broker.
- **Rocket Status** — Mirrors the `rocket/status` MQTT topic (OFFLINE / ONLINE / LAUNCHED / FLIGHT / LOW_BATTERY).
- **GPS Satellites** — Live satellite count from the NEO-6M module on the ESP32.
- **Battery Voltage** — Live battery reading; turns red when below 3.5 V (threshold set in ESP32 firmware).



Figure 7.2: Status bar that appears at the top of dashboard

7.4.2 Telemetry Data Panel

Displays live numerical readings updated at 10 Hz via SocketIO:

- Kalman-filtered altitude (m AGL) and raw barometric altitude.

- Kalman-filtered vertical velocity (m/s).
- 3-axis acceleration vector and magnitude (m/s^2).
- Atmospheric temperature ($^{\circ}\text{C}$) and pressure (hPa).
- WiFi RSSI (dBm).

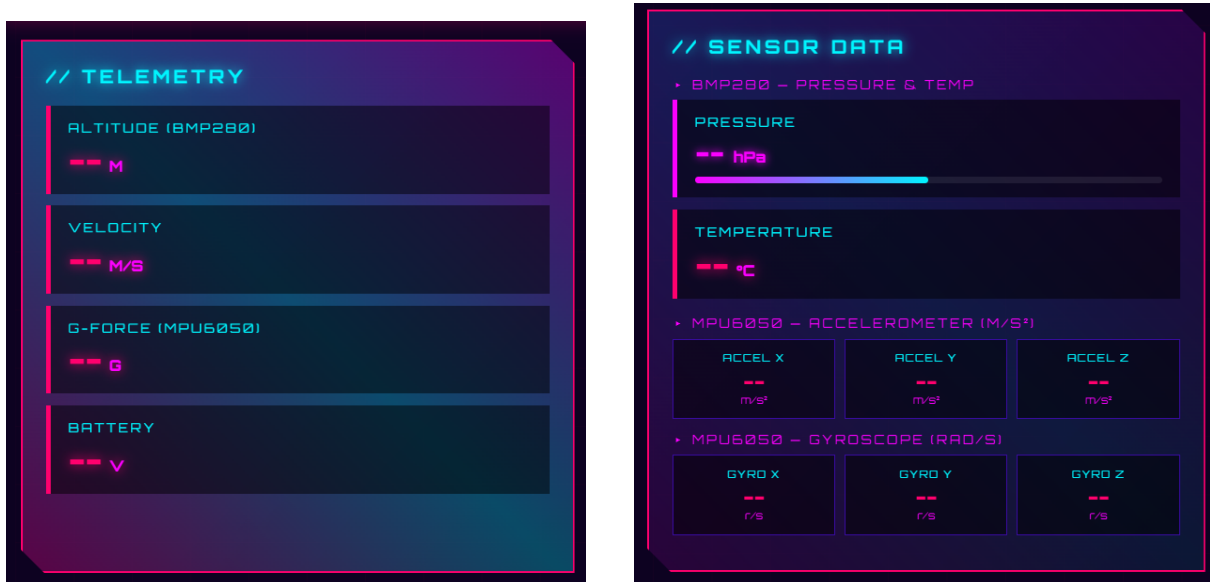


Figure 7.3: Telemetry data panel (left) and detailed sensor data breakdown for the BMP280 and MPU6050 (right), shown in idle state prior to launch

7.4.3 Flight Statistics Panel

Running maximums tracked since last reset: max altitude, max velocity, max acceleration magnitude, and total flight time (launch event to landing event).



Figure 7.4: Flight statistics panel, showing maximum altitude, velocity, G-force, flight time, temperature, and overall flight state (idle prior to launch)

7.4.4 GPS Map

An interactive Leaflet.js map plots the rocket's GPS position in real-time. A red polyline traces the flight path. Ground coordinates and GPS altitude are displayed below the map.

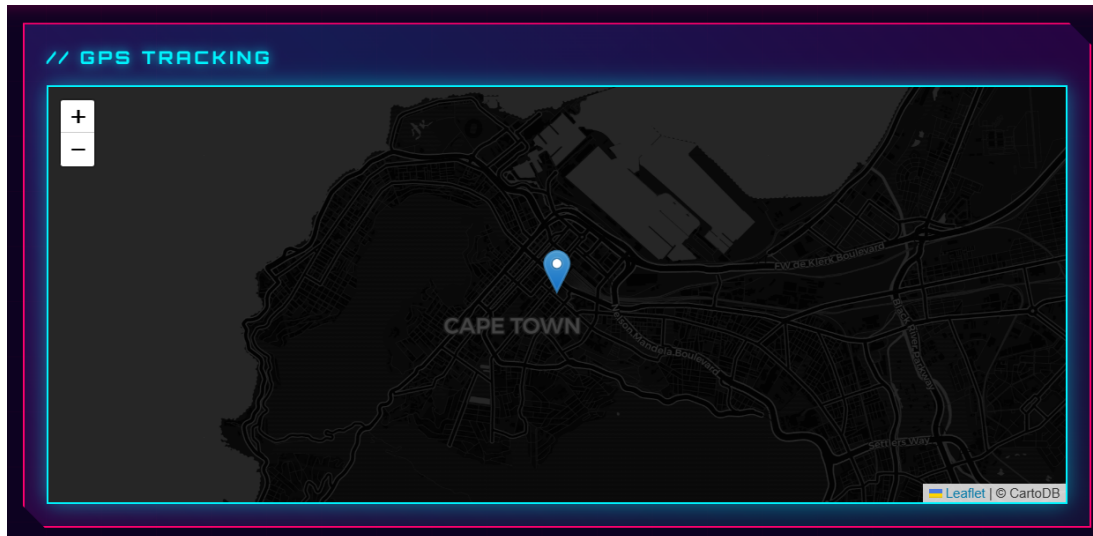


Figure 7.5: GPS tracking panel, showing the live Leaflet.js map centred on the ground station location prior to launch

7.4.5 Altitude and Acceleration Graphs

Two Chart.js line charts display the last 100 data points of:

- Kalman altitude vs. time.
- Total acceleration magnitude vs. time.

Both charts use automatic y-axis scaling and a sliding time window.

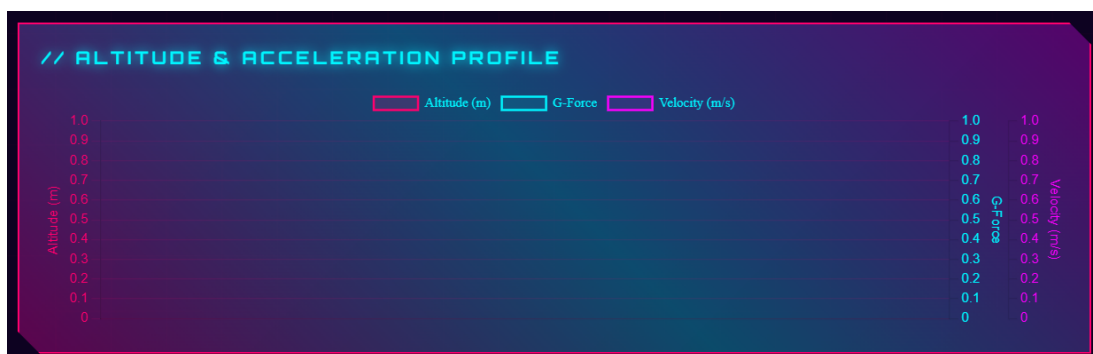


Figure 7.6: Altitude, G-force, and velocity profile chart, shown awaiting telemetry data before launch

7.4.6 Flight Events Log

A chronological list of detected flight events: ONLINE, LAUNCHED, FLIGHT, LOW_BATTERY, and any future apogee/landing events published by the flight computer.

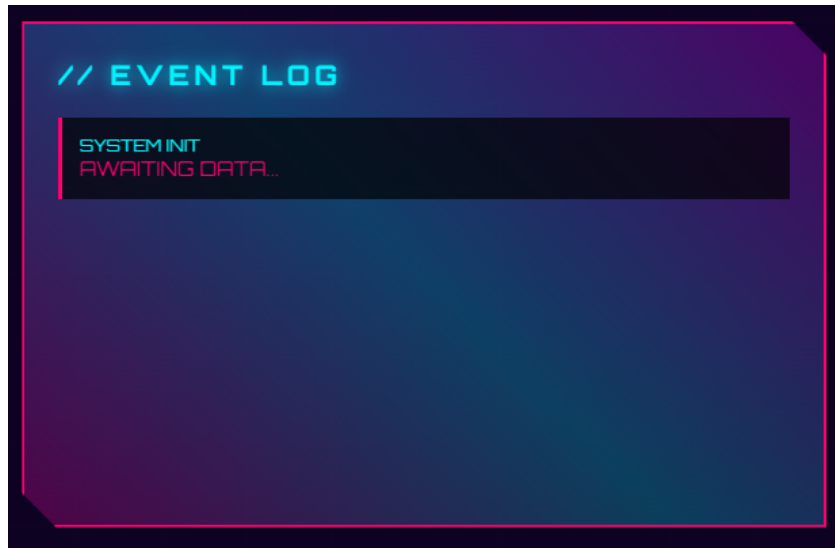


Figure 7.7: Flight events log panel, showing the system initialisation entry prior to data being received

7.4.7 Control Panel

Operator controls available from the dashboard:

- **ARM / DISARM** — Arms or disarms the pyro channel via `rocket/commands/arm` MQTT topic.
- **ABORT** — Emergency stop command.
- **Deploy Parachute** — Manual override for parachute deployment.
- **Start / Stop Logging** — Toggles CSV data recording.
- **Reset Data** — Clears history and stats for next flight.

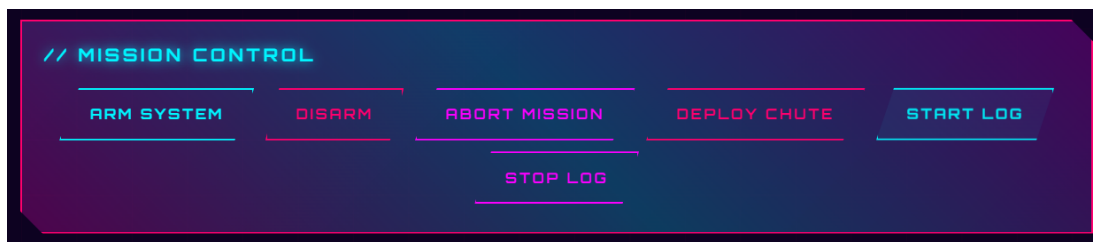


Figure 7.8: Mission control panel, showing the Arm, Disarm, Abort, Deploy Chute, and Start/Stop Log controls

7.5 Network Configuration

7.5.1 WiFi Access Point

The Pi 4 is configured as a WiFi access point using `hostapd` and `dnsmasq`, creating the `ROCKET_TELEMETRY` SSID on the 2.4 GHz radio. The ESP32 in the rocket is configured to connect to this network on startup. Table 7.6 summarises the relevant configuration values (in place of separate firmware and `dhcpcd.conf` listings).

Table 7.6: WiFi and static-IP configuration values

Setting	Location	Value
ssid	ESP32 firmware	ROCKET_TELEMETRY (
password	ESP32 firmware	Secure passphrase
mqtt_server	ESP32 firmware	192.168.43.25 (must match Pi 4 static IP)
wlan0 static_ip_address	Pi 4 /etc/dhcpd.conf	192.168.43.1/24

7.5.2 Mosquitto MQTT Broker Configuration

The broker is configured for anonymous connections on the local network, with QoS 1 persistence and full logging enabled, as summarised in Table 7.7.

Table 7.7: Key settings in /etc/mosquitto/conf.d/rocket.conf

Setting	Value / Purpose
allow_anonymous	true — permits unauthenticated connections on the local network
listener	1883 on 0.0.0.0 — standard MQTT port, all interfaces
persistence	true, stored at /var/lib/mosquitto/ — retains QoS 1 messages
log_dest / log_type	File logging to /var/log/mosquitto/-mosquitto.log, all message types

Chapter 8

Additional

8.1 Bill of Materials

Table 8.1: Bill of Materials with South African pricing (ZAR, incl. VAT where applicable)

Designator	Comment	Footprint / Package	Qty	Unit Price (R)	Total (R)
<i>Flight Computer & Sensors</i>					
U1	ESP32-WROOM-32 development board (Communica BMT ESP-32)	38-pin dev board	1	115.00	115.00
U2	MPU6050 6-axis accelerometer/-gyroscope module (GY-521)	Breakout, I2C	1	60.00	60.00
U3	BMP280 barometric pressure sensor module (GY-BMP280-3.3)	Breakout, I2C/SPI	1	65.00	65.00
U4	NEO-6M GPS module with ceramic antenna	Breakout, UART	1	220.00	220.00
<i>Power Circuit</i>					
U5	MIC5219-3.3YC5-TR 3.3V LDO regulator	SOT-23-5	1	18.00	18.00
BAT1	3.7V 1S LiPo battery, 850mAh, JST-PH connector	Pouch cell	1	180.00	180.00
–	LiPo charger (USB TP4056-based module)	Breakout	1	35.00	35.00
C1–C4	Ceramic capacitors, 100nF	0805/THT	8	1.50	12.00
C5–C7	Electrolytic/tantalum capacitors, 10–100 μ F	THT	3	3.00	9.00
R1–R11	Resistors, assorted values (1k–100k)	THT 0.25W	15	0.80	12.00
<i>Ignition & Pyro System</i>					
Q1	IRLZ44N logic-level N-channel MOSFET	TO-220	1	18.00	18.00
–	Electric matches (e-matches) [†]	–	4	45.00	180.00
–	9V battery (igniter supply, PP3)	–	1	45.00	45.00
SW1	SPST arming switch (key or slide)	Panel mount	1	25.00	25.00
<i>Recovery System</i>					
–	Table cloth plastic (parachute canopy, 1 m ²)	–	1	60.00	60.00
Continued on next page					

Table 8.1 Continued from previous page

Designator	Comment	Footprint / Package	Qty	Unit Price (R)	Total (R)
<i>Propulsion</i>					
–	Estes C6-5 model rocket motor (3-pack, incl. igniters/plugs) [†]	18 mm	4 pack	–	1200.00
<i>Airframe (CAD/3D-Printed & Cardboard)</i>					
–	PLA filament, 1 kg (nose cone, fin jig, motor mount, launch lug)	1.75 mm	1	350.00	350.00
–	Cardboard body tube, 40.4 mm dia. (kraft spiral-wound, 1 m)	–	1	20.00	20.00
–	Launch rod, 3 mm steel rod, 1 m	–	1	55.00	55.00
–	Epoxy/PVA adhesive	–	1	70.00	70.00
<i>Ground Station</i>					
U6	Raspberry Pi 4 Model B, 4GB RAM	SBC	1	2 587.50	2 587.50
–	microSD card, 32GB (Class 10/A1)	–	1	150.00	150.00
–	USB-C power supply, 5V/3A	–	2	180.00	360.00
<i>Consumables & Miscellaneous</i>					
–	Resistors	–	–	–	20.00
–	Prototyping/vero board	–	2	45.00	90.00
Subtotal (excl. shipping/import duty)					6 356.50
Estimated shipping/courier (multiple suppliers)					350.00
TOTAL (ZAR)					6 706.50

8.2 Acceptance Test Results

ID	Description	Result	Measured Values
AT-HW01	Ascent trajectory	PASS	Apogee 89.48 m; max velocity 33.71 m/s; max acceleration 6.74 G (≤ 15 G requirement).
AT-HW02	Static stability margin	PASS	Margin 1.52 cal (initial) to 1.60 cal (burnout); within the required 1–2 cal band throughout.
AT-HW03	Recovery descent, impact, wind drift	PASS	Terminal velocity 2.46 m/s; impact deceleration 5.01 G (≤ 20 G limit); 95th-percentile landing radius 420.9 m (≤ 500 m).
AT-HW04	Structural safety factors	PASS	Peak thrust load 14.1 N; retention SF 2.84 (≥ 2); buckling SF 512.67 (≥ 1.5).
AT-SW01	Sensor sampling rate	PASS	BMP280 19.53 Hz (n=1172); MPU6050 48.82 Hz (n=2929); both exceed the 10 Hz requirement.
AT-SW02	Apogee detection	PASS	FSM ejection triggered at 7.41 s, 1.34 s <i>before</i> the physical peak altitude at 7.41 s; 0 false pre-trigger detected during the run (ASCEND/EJECT/DESCEND/-STANDBY cycle repeated twice within the 20 s window).
AT-SW03	Onboard memory backup	PASS	999 of 999 pre-loss points recovered after simulated power loss (USB disconnect/reconnect); zero points missing.
AT-SW04	Telemetry latency	PASS	100/100 packets received (0.0% loss); median one-way latency 8.14 ms; 95th-percentile 8.66 ms (< 500 ms requirement).
AT-SW05	Data logging format	PASS	16/16 schema columns populated across 500 downloaded telemetry rows; zero malformed rows.

Table 8.2: Acceptance Test Results (test session of [insert date])

8.3 System Integration and Interfacing

Interfacing Table

Interface	Description	Pins / Output
ESP32 ↔ MPU6050	3-axis accel + 3-axis gyro data (I2C)	SDA: GPIO21, SCL: GPIO22, Addr: 0x68
ESP32 ↔ BMP280	Barometric pressure/temperature/altitude (I2C, shared bus)	SDA: GPIO21, SCL: GPIO22, Addr: 0x76
ESP32 ↔ NEO-6M GPS	Position, altitude, satellite count (UART, NMEA)	RX2: GPIO16, TX2: GPIO17
ESP32 ↔ IRLZ44N MOSFET	Igniter gate drive (digital output)	GPIO5 (gate, via R11 20 kΩ)
Battery ↔ ESP32 ADC	Battery voltage monitoring (analog input, via divider)	GPIO34 (ADC1_CH6)
Battery ↔ BT1 (Igniter rail)	Direct igniter/pyro supply (bypasses LDO)	V_{BAT+} / V_{BAT-}
ESP32 ↔ Raspberry Pi 4	Telemetry uplink (WiFi 802.11n, MQTT over rocket/#)	Wireless (2.4 GHz)
Raspberry Pi 4 ↔ Mosquitto Broker	Local message routing (loop-back MQTT)	localhost:1883
Raspberry Pi 4 ↔ Browser Dashboard	Real-time telemetry push and REST control	HTTP/WebSocket, port 5000
Ground Station ↔ Operator	ARM/DISARM/ABORT/DEPLOY commands	WebSocket/commands/arm (MQTT publish)

Table 8.3: System interfacing specifications

8.4 Discussion and Considerations

8.4.1 BMP280 Failures and Shortcomings

During testing, the BMP280 produced consistently incorrect altitude readings. This was traced to its reliance on a manually supplied sea-level reference pressure (P_0) for altitude calculation:

$$h = 44330 \times \left(1 - \left(\frac{P}{P_0} \right)^{\frac{1}{5.255}} \right)$$

A fixed or estimated P_0 was used rather than the actual local sea-level pressure at the launch site, and since even a 1–2 hPa error in P_0 translates to roughly 8–16m of altitude error, this produced a consistent offset rather than random noise. The result was an incorrect ground-level baseline that propagated through the entire flight profile, making it unreliable for verifying the target apogee ($500\text{m} \pm 50\text{m}$) against the design specification.

This was not a hardware or noise limitation, but a calibration issue, correcting it would require dynamically calibrating P_0 at the pad immediately before launch rather than using a fixed value. This inaccuracy was a key

factor in moving to the SPL06-001 (Section 4.2.2) alongside a more rigorous pre-launch calibration procedure.